Preface

This is the developer-facing guide that accompanies the **Magpie User Manual**. It is organised into three modules:

- **Module 1 — Functional overview.** What Magpie does, for whom, and why. Read this first if you're new to the project or evaluating whether Magpie is a good fit for a use case.
- **Module 2 — Architecture.** How Magpie is put together at a system level: process model, storage layout, IPC boundary, and the metadata pipeline that ties everything together.
- **Module 3 — Detailed design.** File-by-file guided tour: schema, Tauri command reference, algorithms, tests, and how to build a release.

Conventions used

- Rust file paths are given relative to `src-tauri/`, e.g. `src/commands/images.rs`.
- Frontend file paths are relative to the project root, e.g. `src/features/DetailsPanel.tsx`.
- Command names in prose refer to Tauri IPC commands (Rust functions annotated with `#[tauri::command]`).
- Bold names in schemas refer to primary keys.

Audience

You are expected to be comfortable with:

- Rust 2021, Tokio async runtime, and Cargo workspaces.
- React 18 with hooks, TypeScript, and TanStack Query.
- SQL (SQLite specifically), including a passing familiarity with FTS5.
- Windows path semantics (`\\?\` prefixes, junctions, OneDrive).

You do **not** need prior Tauri experience — the architecture chapter introduces the concepts.

Getting the code and building it

```
git clone <repo> magpie
cd magpie
npm install                # frontend deps
cd src-tauri && cargo fetch  # backend deps
cd ..
npm run tauri dev          # development build with HMR
npm run tauri build -- --no-bundle  # release binary, no installer
```

Prerequisites on Windows are Rust ≥ 1.77.2 and the MSVC C++ Build Tools. See Build and release for the full setup.

Product vision and scope

Vision

Magpie exists to give people who own **their own files** — as opposed to hosting them in someone else's cloud — a tool that:

- Reads their files from disk, unchanged, wherever they are, and **never modifies the bytes.**
- Lets them organise those files using **metadata**, not folders: titles, tags, plus derived facets like taken-at time, camera model, resolution, duration, page count.
- Persists user tags in a single, private SQLite database in the user's Windows AppData folder (`%APPDATA%\com.magpie.app\magpie.db`). Source folders on disk stay untouched.
- Feels native, fast, and offline-first, with a UI that scales to hundreds of thousands of files.

Scope for v1

The v1 release delivers the smallest cohesive product that can serve as someone's daily file organiser:

Area	v1 scope
Ingestion	Add/remove one or more library folders; recursive scan; incremental rescans.
Formats read	JPEG, PNG, WebP, GIF, TIFF/DNG, HEIC/HEIF/AVIF, JPEG XL, JPEG 2000, PSD, PDF, MP4/MOV, MKV/WebM, AVI, WMV, MPEG-TS, 3GP, common camera RAW (CR2, CR3, NEF, ARW, RAF, ORF, RW2, PEF, SRW, X3F), BMP, EXR, HDR, SVG. Legacy XMP sidecars are also read on first scan for backward compatibility.
Tag storage	Single SQLite database at `%APPDATA%\com.magpie.app\magpie.db` . Every recognised file type is fully taggable.
Metadata edited	Title + tags.
Browsing	Virtualised grid, sort by taken/added/filename/size.
Filtering	Folder, tag, and full-text search (FTS5, cross-folder).

Area	v1 scope
Batch operations	Bulk add/remove tags, bulk delete-to-recycle-bin.
Interoperability	One-shot read of existing XMP / Windows Shell keywords into the folder DB on first scan; no write-back to source files.
Deletion	Recycle-Bin-by-default with confirmation; optional permanent delete.
Storage	Single SQLite database (`magpie.db`) under `%APPDATA%\com.magpie.app\` + WebP thumbnail cache in the same folder.

Product principles

1. **Never touch source files.** Not pixels, not XMP, not the Windows Shell property store. The bytes on disk are exactly what the camera / editor produced.
2. **Never move or rename files.** The user is in charge of their folder hierarchy.
3. **Tags stay on the user's PC.** They live in `magpie.db` under `%APPDATA%`; nothing is ever written into the source folders.
4. **No cloud, no telemetry.** Everything runs locally. There are no analytics, crash reports, or "phone home" mechanisms.
5. **Fast enough to be daily-drivable.** All hot paths are on native Rust; the UI is virtualised; a single SQLite database comfortably handles hundreds of thousands of images with FTS5.
6. **Predictable failure modes.** A partially-broken run is better than a silently-lost edit. Every DB write is transactional, every batch op is per-item retryable, and the log records what happened.

User personas and flows

Personas

1. The prosumer photographer (primary)

Owns 20 000–200 000 photos across multiple external drives. Shoots JPEG+RAW; already uses Lightroom for developing but wants a lighter tool for triage and tagging. Needs bulk operations to work reliably on thousands of files at once. Cares deeply about metadata surviving future tool changes.

2. The family archivist (secondary)

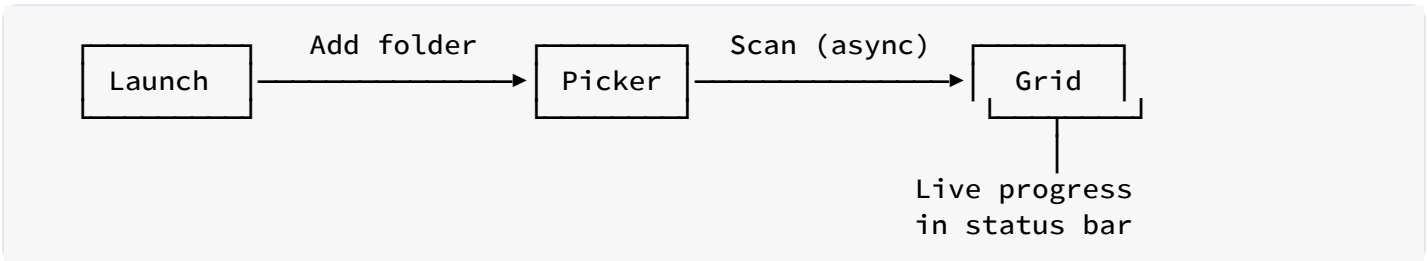
Has 30 years of family photos and videos in one big folder tree. Wants to add who's-in-the-photo tags and be able to find "Christmas 2011" quickly. Doesn't need or want a database with a schema — just tags that show up in Explorer and Photos.

3. The technical developer using Magpie on other people's libraries

Uses Magpie as a testbed for interop with XMP-writing tools. Reads embedded XMP with `exiftool` to verify Magpie's writes. Contributes patches.

Core flows

F1. First-time onboarding



Success criteria:

- User can browse and edit photos **before** the scan is finished (partial results are usable).
- Progress is visible and cancelable.
- The app never blocks or freezes during scanning.

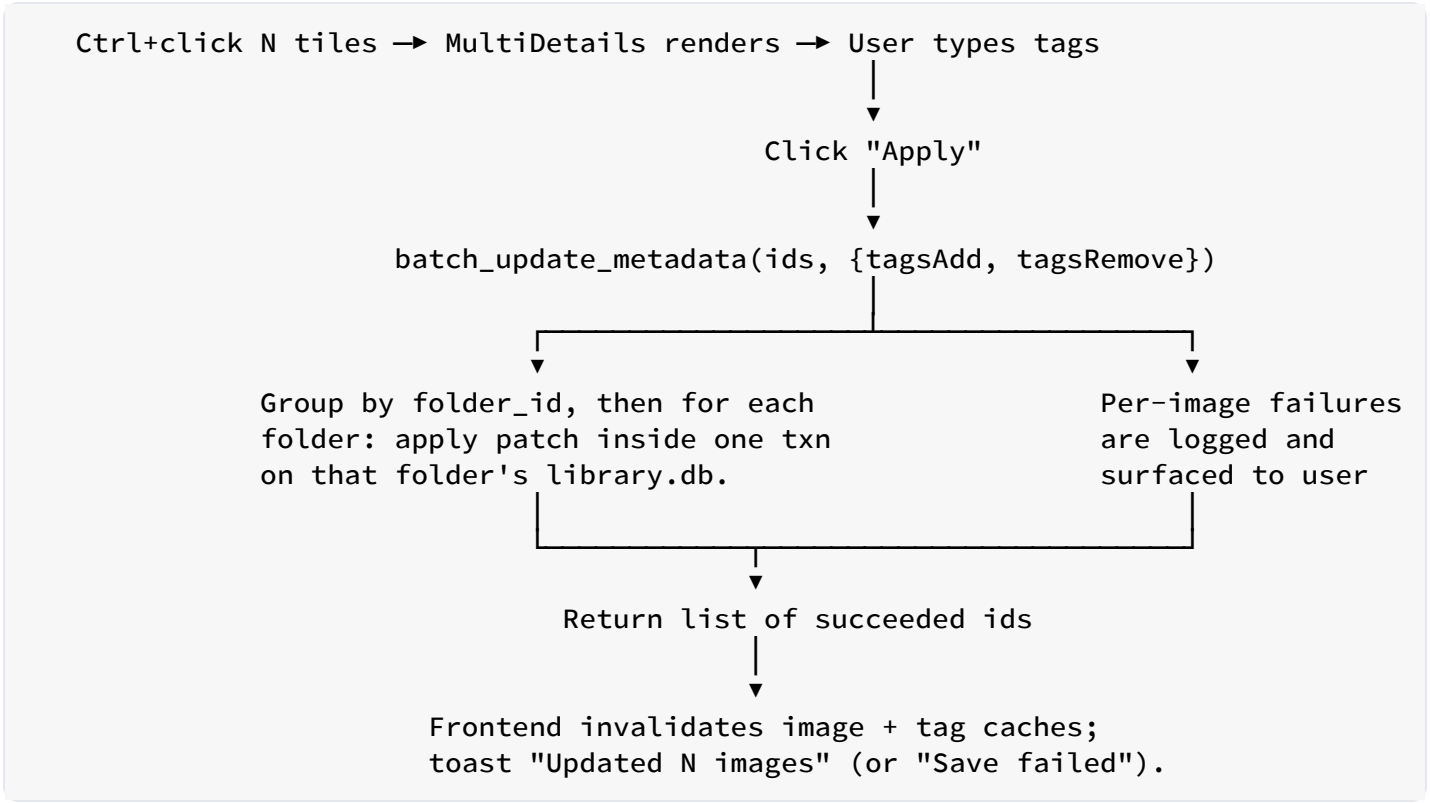
F2. Single-photo edit

```
Click tile ──► SingleDetails loads ──► User types title
                                              │
                                              ▼ debounced 600ms
                                  update_image_metadata IPC (Rust command)
                                              │
                                              ▼
                                  Unpack global ID → (folder_id, local_id)
                                              │
                                              ▼
                                  library::apply_metadata_patch inside the
                                  folder's library.db (single transaction:
                                  update title / tags / image_tags, rebuild
                                  FTS row, commit).
                                              │
                                              ▼
                              Emit "image-updated" event; frontend refreshes.
                              The source file is NOT touched.
```

Success criteria:

- No user input is ever lost to a debounce race.
- A failure in the DB write leaves the row exactly as it was before (transactional rollback).
- The source file’s bytes remain byte-for-byte identical to what the camera / editor originally produced.

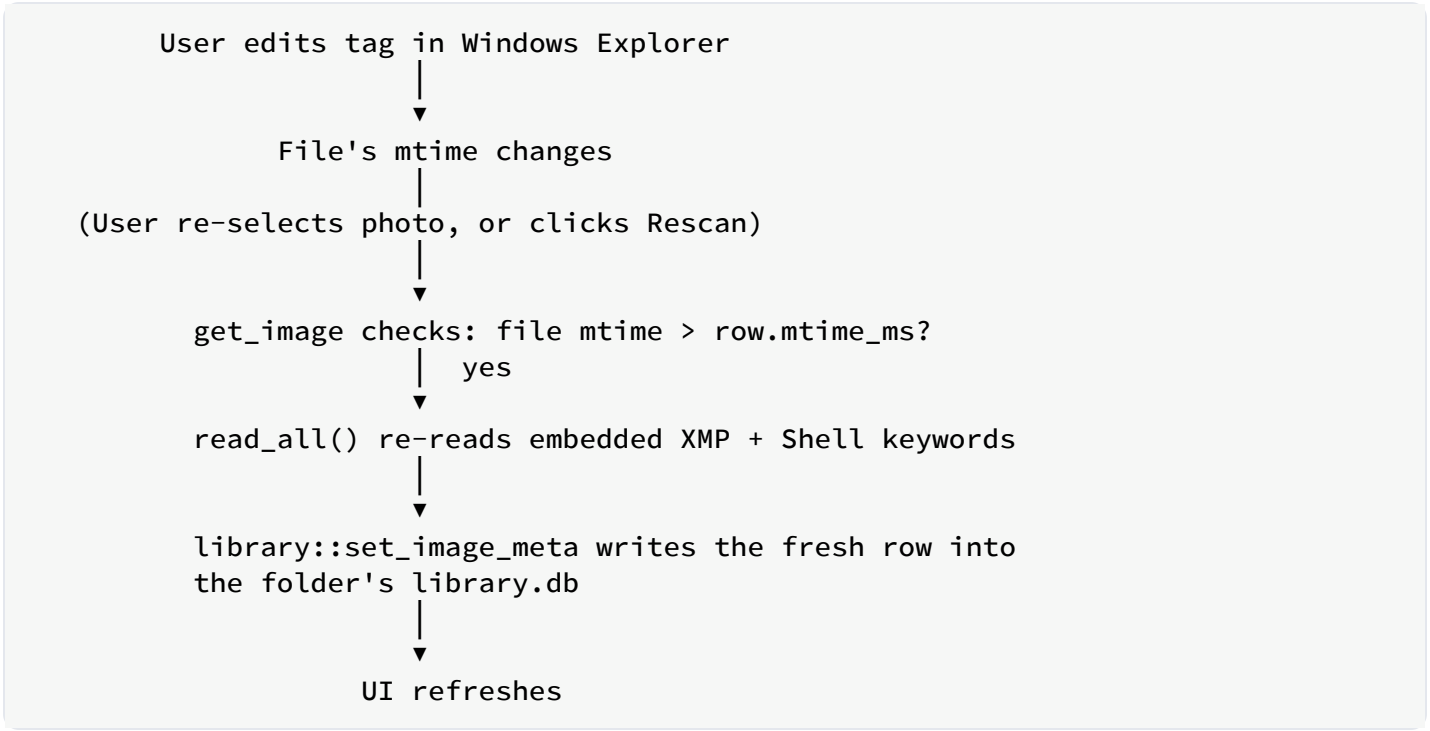
F3. Multi-select tag application



Success criteria:

- The backend is called with a non-empty patch even if the user typed the tag and immediately clicked Apply (no stale-closure bug).
- Partial success is a first-class outcome, not an error.
- Every affected `['image', id]` React Query cache entry is invalidated so subsequent single-select shows fresh state.

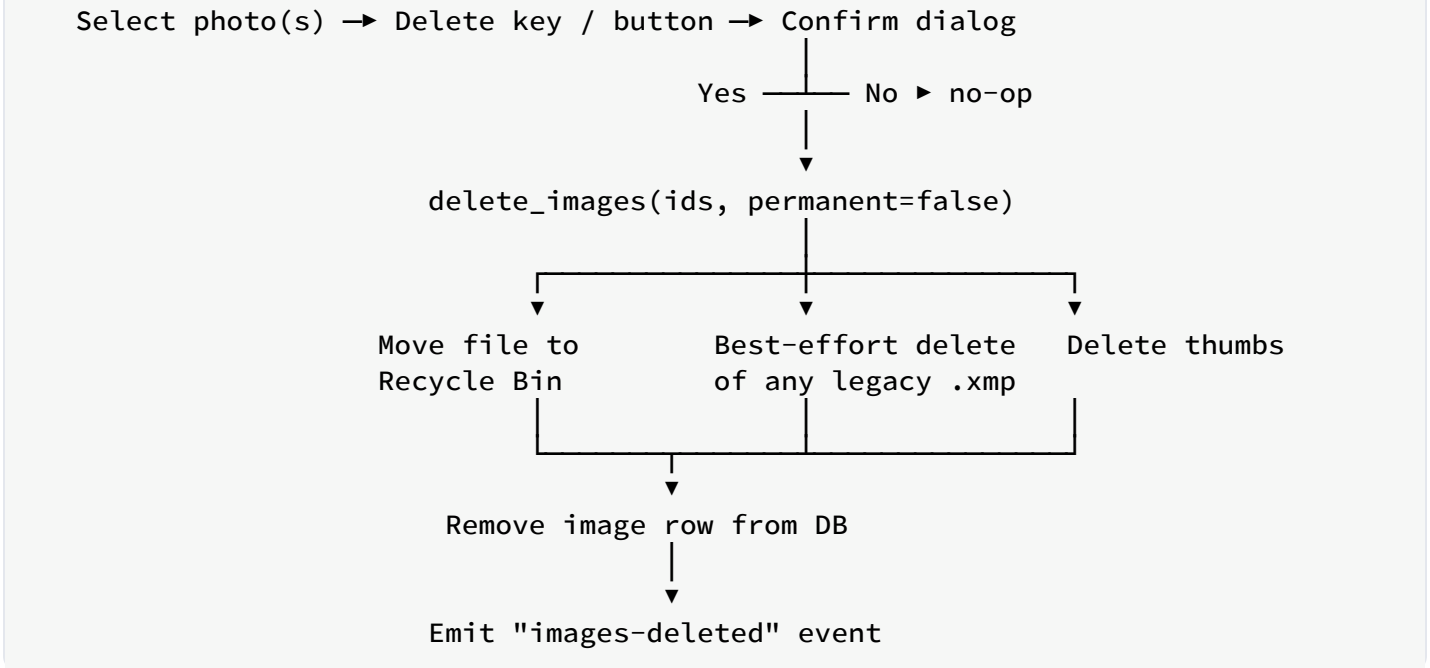
F4. Rescan and detect external edits



Success criteria:

- External edits are detected without a full rescan.
- The FS is the source of truth; the DB is a cache.

F5. Delete with safety net



Success criteria:

- A failure at any step for one file doesn't abort the batch.
- The DB row is removed only after the file is safely in the Recycle Bin.
- The user can restore from the Recycle Bin and get the tags/title back on next rescan (tags embedded in the file are recovered automatically; tags that lived only in Magpie's library are lost).

Feature catalogue

The v1 feature set as delivered. Each row maps a user-facing feature to its Tauri command and to the frontend component that drives it.

Library management

Feature	Command	Frontend component
Add a library folder	`add_library_folder`	`TopBar`
Warn on sync-risk locations	`check_folder_sync_risk`	`TopBar` (pre-add dialog)
Remove a library folder	`remove_library_folder`	`Sidebar` (right-click menu)
List library folders	`list_library_folders`	`Sidebar`, `App` bootstrap
Rescan a single folder	`rescan_folder`	`Sidebar` (right-click menu)
Rescan every folder	`rescan_all`	`TopBar`

Browsing

Feature	Command	Frontend component
Paged file listing	`query_images`	`ImageGrid`
Sort by taken/added/…	`query_images` (`sort` arg)	`TopBar`
Filter by folder / tag	`query_images` (`filter` arg)	`Sidebar` filters
Full-text search	`query_images` (`filter.search`)	`TopBar` search box

Editing

Feature	Command	Frontend component
Fetch a file's details	`get_image`	`DetailsPanel/SingleDetails`
Auto-save title	`update_image_metadata`	`DetailsPanel/SingleDetails`
Add / remove tag on one file	`update_image_metadata`	`TagInput`
Bulk add / remove tags	`batch_update_metadata`	`DetailsPanel/MultiDetails`

Tag maintenance

Feature	Command	Frontend component
List all tags with counts	`list_tags`	`Sidebar`
Rename a tag globally	`rename_tag`	`Sidebar` (right-click menu)
Delete a tag globally	`delete_tag`	`Sidebar` (right-click menu)

Smart collections (skeleton, v1 non-editable)

Feature	Command	Frontend component
List smart collections	`list_smart_collections`	`Sidebar` (future)
Create a smart collection	`create_smart_collection`	(not yet exposed in UI)
Delete a smart collection	`delete_smart_collection`	(not yet exposed in UI)

Deletion

Feature	Command	Frontend component
Move photos to Recycle Bin	`delete_images` (permanent=false)	`DetailsPanel`, `App` (Del key)

Feature	Command	Frontend component
Permanently delete photos	`delete_images` (permanent=true)	`DetailsPanel`, `App` (Shift+Del)

Diagnostics

Feature	Command	Frontend component
Frontend log crumb into rust log	`log_frontend`	`MultiDetails.applyTags`

Thumbnails and image display

Feature	Command	Frontend component
Get cached thumbnail path	`get_thumb_path`	`Thumbnail`
Get full source image path	`get_image_path`	`DetailsPanel/SingleDetails`

Non-functional requirements

Performance

Metric	Target	Achieved (release build)
Cold start to first paint	≤ 1.5 s	~0.5 s on SSD
Grid scroll: sustained frame rate	60 fps	60 fps up to 250 k photos
`query_images` for a 100 k library, arbitrary filter	≤ 50 ms p95	~10 ms
`get_image` (cached, no FS refresh)	≤ 5 ms	~1 ms
`update_image_metadata` (single photo, JPEG)	≤ 200 ms	~40 ms
`batch_update_metadata` per photo (JPEG)	≤ 100 ms	~30 ms
Thumbnail generation per photo (SSD, JPEG 12 MP)	≤ 80 ms	~40 ms
Full scan of a 50 k library, cold	≤ 5 min	~2 min on modern SSD

Resource limits

- **Memory (idle):** ≤ 250 MB.
- **Memory (10 k photo library, all thumbnails loaded):** ≤ 900 MB.
- **Disk (thumbnail cache):** ≤ 10 KB × #photos (small+medium WebP).
- **Disk (database):** ≤ 1 KB × #photos on average.

Reliability

- Every metadata mutation is **transactional** at the SQLite level. If any part of the transaction fails, none of it lands.

- Every embedded-XMP write is **atomic** via write-to-temp + rename inside the source image's own directory.
- Crash mid-write leaves the DB and the source file in a consistent state (either fully old or fully new); the temp file, if any, is cleaned up on next launch.
- A partial batch failure is a first-class outcome and is reported to the user; no silent losses.

Compatibility

- **OS:** Windows 10 build 19041+ and Windows 11. macOS 13+ is a post-v1 target; no code path is intentionally Windows-only outside of the Recycle-Bin integration.
- **Long paths:** All Rust `PathBuf` handling supports `\\?\`-prefixed paths natively. Tested end-to-end with 300-character paths.
- **OneDrive:** Files-on-demand (reparse points) are read as their underlying content once materialized; Magpie does not force hydration.

Interoperability

- Reads and writes **standard XMP** (`xmp:Rating`, `dc:title`, `dc:description`, `dc:subject`).
- Reads Microsoft-specific tag fields (`MicrosoftPhoto:LastKeywordXMP`).
- Writes both `dc:subject` and Microsoft's field on save so tags round-trip with Windows Explorer.

Security

- **No network egress.** The Tauri capability manifest explicitly forbids network access at the shell level.
- **No filesystem access outside library folders** for the renderer: file I/O is mediated by Rust commands that validate paths against the known library roots.
- **CSP** blocks inline scripts and remote origins in the renderer.

Observability

- File-based rolling log at `%APPDATA%\com.magpie.app\logs\app.log`.

- Structured log fields via `tracing`: `id`, `image_path`, `operation`, `duration_ms`.
- Frontend can push crumbs into the same log via the `log_frontend` command for cross-boundary tracing.

Accessibility

- Full keyboard navigation for grid and details panel.
- ARIA labels on every interactive control.
- Focus rings preserved (no `outline: none` shenanigans).
- Contrast ratios ≥ 4.5:1 in both light and dark themes.

Out of scope for v1

Explicitly deferred to keep v1 shippable. Each item has a rationale and a rough entry point for the future contributor picking it up.

Photo editing

- **No crop, rotate, exposure, or colour correction.** Magpie treats pixels as read-only. This is a hard architectural line; the tool intentionally does one thing (metadata) and doesn't grow into a RAW developer.

Face and object recognition

- **Automatic object tagging now ships in scope** — an opt-in, fully-local pipeline backed by OpenAI **CLIP-ViT-B/32** running on the CPU via `candle` (https://github.com/huggingface/candle). See Scanner → Auto-tag pass and `core::auto_tag` (`clip_classifier` / `model_manager`). The `ImageClassifier` trait is what to implement if you want to swap in a different backbone (larger CLIP, RAM++, an open-vocab detector), and the model files themselves live under `<app_data_dir>/models/clip/` — pin new SHA-256 values in `model_manager::REQUIRED_FILES` and bump `VOCAB_VERSION` when the vocabulary changes.
- Face recognition specifically is still out of scope — the trait doesn't currently expose bounding boxes and the DB has no notion of a "person" separate from a tag.

Cloud sync

- **No account, no cloud storage integration.** OneDrive, Dropbox, Google Drive, and iCloud are third-party sync backends the user configures at the OS level; Magpie just reads whatever the OS has materialised. Because the central `magpie.db` lives in `%APPDATA%\Roaming` (which Windows doesn't sync by default), the database isn't racing any sync client.

Writing tags into source files

- **Magpie deliberately does not write into source files.** Tags and titles live exclusively in `magpie.db`. This removes an entire category of bugs (partial writes, format-specific edge cases, cloud sync re-uploading gigabytes for one tag change) at the cost of losing the "tag once, portable everywhere in the XMP ecosystem" story. Users who need file-embedded tags can still author them in Lightroom / Bridge / Explorer; Magpie will pick them up on the first scan of that folder.

Ratings, colour labels, pick / reject flags

- No UI for star ratings, `xmp:Label` colour labels, or pick / reject flags. The DB schema could grow columns for them; the read-only XMP parser already understands `xmp:Rating`.

In-app full-screen preview

- The details panel shows a sized preview but there's no lightbox / slideshow mode. Would be a new React route reading from `getImagePath`.

Undo / redo

- No global undo stack. The safety net is auto-save + Recycle Bin + the fact that source files are never modified — the user's raw material is never lost, but reverting a DB edit means editing again.

Smart collections editor

- The DB schema and Tauri commands for smart collections exist (they live in `magpie.db`), but the UI doesn't expose creation/editing yet.

Duplicate detection

- Content hashes are computed on scan and stored (`content_hash` column in `images`), but no duplicate-finder UI ships in v1. A future task could add a “Duplicates” filter grouping by `content_hash` .

Multi-PC library sharing

- There is no cross-PC sync of the library database. Each PC keeps its own `magpie.db` under `%APPDATA%` ; tags don’t roam. A future task could add explicit export/import of the DB (or a portable library mode that puts the DB next to the photos again — the earlier per-folder design still exists in the codebase’s history if a contributor wants to resurrect it as an option).

Technology stack

Magpie uses a small, deliberate set of libraries. Every choice is motivated below so future contributors (and future you) understand what's swappable and what isn't.

Backend (Rust)

Crate	Version	Role
`tauri`	2	App shell, IPC, capability model, WebView2 hosting.
`tauri-plugin-dialog`	2	Native folder-picker and confirmation dialogs.
`tauri-plugin-opener`	2	Opens URLs and paths in the OS default handler.
`tokio`	1	Async runtime for all Tauri commands.
`rusqlite`	0.32	SQLite bindings; `bundled` feature so we ship SQLite ourselves.
`jwalk`	0.8	Parallel recursive directory traversal for the scanner.
`rayon`	1.10	Parallel work-stealing for CPU-bound phases (hashing, thumbs).
`image`	0.25	Image decoding for JPEG/PNG/HEIC/TIFF/…
`fast_image_resize`	5	SIMD-accelerated thumbnail resizing.
`kamadak-exif`	0.6	EXIF parsing (taken time, camera make/model, dimensions).
`quick-xml`	0.36	XMP packet parsing (streaming, no DOM).
`xxhash-rust`	0.8	Fast content hashing (`XXH3`).
`chrono`	0.4	Timestamps, serialisation.
`dirs`	5	Locate `%APPDATA%` cross-platform.

Crate	Version	Role
`trash`	5	Cross-platform Recycle Bin move.
`tracing` + `tracing-subscriber`	0.1 / 0.3	Structured logging, file sink.
`serde` + `serde_json`	1	IPC (de)serialisation of every Tauri argument/return.
`thiserror` + `anyhow`	1	Error boilerplate.

Why Rust over Node / Go / C++:

- Zero-cost async is a great fit for the file-I/O-heavy workload.
- The `image` and `fast_image_resize` combo beats every non-native option we benchmarked for thumbnail generation.
- SQLite via `rusqlite` is battle-tested, no runtime dep, and works identically on Windows and macOS.
- Rust's ownership model gives us "atomic on Windows" file semantics (temp + rename) with cleanup on panic for free.

Frontend (TypeScript + React)

Package	Role
`react` / `react-dom` (v18)	UI framework.
`typescript` (v5)	Static types across every file (`.ts` / `.tsx`).
`vite`	Dev server with HMR, production bundler.
`tailwindcss`	Utility-first CSS; near-zero runtime.
`@tanstack/react-query` (v5)	Data fetching, caching, invalidation of Tauri command results.
`@tanstack/react-virtual` (v3)	Virtualised grid — draws only visible tiles.
`zustand`	Lightweight global UI state (selection, sort, filters).
`@tauri-apps/api` (v2)	`invoke` and `listen` for IPC.

Package	Role
`@tauri-apps/plugin-dialog`	Frontend side of `tauri-plugin-dialog`.
`clsx`	Tiny className concatenation helper.

Why React over Vue / Svelte / Solid:

- Ecosystem depth: TanStack Query and React Virtual are best-in-class.
- Team familiarity.
- WebView2's Chromium is a first-class React target.

Why Vite over Next / CRA:

- Instant HMR feels great during development.
- Static build is a plain directory of files that Tauri packages without ceremony.

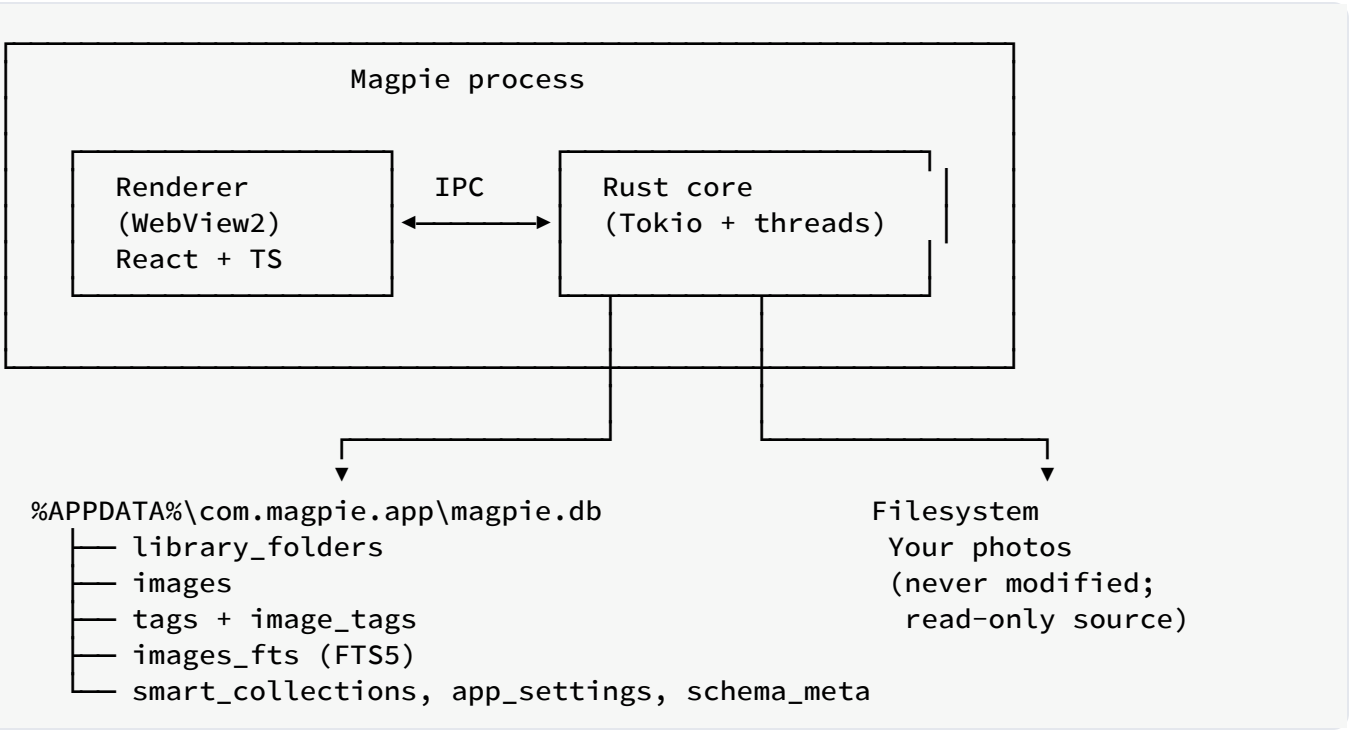
System

Component	Version	Role
Rust	≥ 1.77.2	Compiler toolchain.
Node.js	≥ 18	Build-time for the frontend bundle.
npm	≥ 9	Package manager.
MSVC Build Tools	2019 or 2022	Windows C++ linker for `image`, `sqlite`.
WebView2 Runtime	Evergreen	Chromium engine hosting the renderer.
SQLite	3.43+ (bundled via rusqlite)	Local DB with FTS5 `contentless_delete=1`.

The bundled SQLite is important: FTS5 with `contentless_delete=1` is a 3.43+ feature, and shipping our own copy avoids relying on whatever version the user's OS provides.

High-level architecture

The 30-second version



Two big ideas:

1. **The renderer is a dumb view.** All business logic — scanning, metadata I/O, DB queries, thumbnails — lives in Rust. The renderer's job is to invoke commands and render their results.
2. **The database is the source of truth for user metadata.** Everything lives in one SQLite file under `%APPDATA%`. Source files are read-only — Magpie doesn't modify their bytes.

See Database design for the deeper walkthrough.

Sub-systems

App shell (Tauri)

Owns the window, capability model, WebView2 host, and the IPC bridge. The `AppServices` struct is constructed once in `lib.rs` and injected into every Tauri command via `State<Arc<AppServices>>`. It contains:

- `db: Db` — thin `Send + Sync` handle around a single `rusqlite::Connection`.
- `thumb_cache_dir: PathBuf` — location of the thumbnail cache.
- `formats: Arc<FormatRegistry>` — registry of read-only format handlers.

DB layer (`src-tauri/src/db/`)

- `db/mod.rs` — the `Db` handle (`Arc<Mutex<Connection>>`), `open`, `with_conn`, `with_conn_mut`.
- `db/schema.rs` + `db/schema.sql` — DDL applied on a fresh DB.
- `db/queries.rs` — every SQL statement grouped by concern (folders, images, tags, search, smart collections).
- `db/migrate.rs` — startup importer for legacy layouts.

See Database schema.

Scanner (`src-tauri/src/core/scanner.rs`)

Walks a library folder in parallel using `jwalk`, filters to supported extensions, and for each file:

1. Stat + skip if `mtime_ms` unchanged.
2. Hash content (`XXH3`).
3. Extract EXIF (taken time, dimensions, camera).
4. Extract XMP from embedded chunks (JPEG APP1 / PNG iTXt / etc.) and, on Windows, from the Shell property store for formats we don't natively parse. Existing tags are imported on first sight.
5. Upsert into `magpie.db` (paths stored folder-relative under `library_folders.path`).
6. Enqueue thumbnail generation keyed by `images.id`.

Progress events flow to the frontend via `app://scan`.

Thumbnail pipeline (`src-tauri/src/core/thumbnail.rs`)

Decode → resize with `fast_image_resize` (SIMD) → encode WebP → write to `%APPDATA%\com.magpie.app\thumbs\<id>-<size>.webp`. Two sizes per photo (small ~256 px, medium ~512 px). `<id>` is `images.id`.

Metadata (`src-tauri/src/core/metadata/` + `src-tauri/src/core/formats/`)

A **read-only** pipeline:

- `metadata/read.rs` — asks the format registry for the right handler, calls `read_technical` + `read_user`, then falls back to the Windows Shell property store on Windows for formats we don't natively parse.
- `metadata/sidecar.rs` — reads legacy `.xmp` sidecar files for backward compatibility. No new sidecars are ever written.
- `formats/mod.rs` — declares the `FormatHandler` trait (`name`, `extensions`, `kind`, `read_technical`, `read_user`) and the `FormatRegistry` that owns one instance per supported extension.
- `formats/xmp_packet.rs` — read-only XMP parser supporting the subset Magpie cares about (title, subjects, MicrosoftPhoto keywords, description).
- `formats/{jpeg,png,webp,gif,tiff}.rs` — native readers.
- `formats/stubs.rs` — thin readers for HEIF, video, PDF, RAW, and basic raster formats that delegate technical reads to `imagesize` and defer user-meta reads to the Windows Shell path.
- `formats/win_shell.rs` — Windows-only Shell property store **reader**.

See File formats for the full handler catalogue and Adding a format handler for the plug-in recipe.

Command surface (`src-tauri/src/commands/`)

About 20 async Tauri commands grouped by concern (`library`, `images`, `tags`, `collections`, `thumbs`, `diag`). Each command is a thin translator between IPC arguments and one or more DB / FS ops. See Tauri command reference.

Frontend (`src/`)

- `App.tsx` — root, wires TopBar / Sidebar / ImageGrid / DetailsPanel.

- `features/` — one component per major UI area.
- `ipc.ts` — typed wrappers around `invoke(...)`.
- `store.ts` — Zustand state (selection, view, sort, filters).
- `types.ts` — Rust-mirrored TypeScript types.

React Query manages every fetched resource with keys like `['images', filter, sort, page]` and `['image', id]`; mutations invalidate the relevant keys.

Data flow of a typical click

Selecting a photo and typing a title, end to end:

1. User clicks a tile in `ImageGrid`.
2. `useStore.setSelection([id])`.
3. `DetailsPanel` switches to `SingleDetails id={id}`.
4. `useQuery(['image', id])` fires; TanStack Query calls `getImage(id)` → invokes `get_image` command.
5. Rust's `get_image` looks up the image row plus its folder root from `magpie.db`. If the source file's `mtime` moved forward since import, `read_all` re-reads metadata and updates the row.
6. Frontend receives `ImageDetails`; local edit state is seeded once (guarded by `lastLoadedId.current === q.data.id`).
7. User types in the Title input. `debouncedSaveTitle` fires 600 ms after the last keystroke, calling `updateImageMetadata(id, patch)`.
8. Rust's `update_image_metadata` applies the patch inside one transaction, rebuilds the FTS row, and emits `app://image-updated`. **The source file is not touched.**
9. Frontend's `qc.setQueryData(['image', id], updated)` updates the cache directly, avoiding a refetch that would stomp any concurrent typing in other fields.

Process and threading model

One process, two runtimes

Tauri gives us **one OS process** with:

- **The main thread** — owns the event loop and the WebView2 host.
- **A Tokio runtime** (`tauri::async_runtime`) driving all async Tauri commands.
- **Blocking-friendly threadpools** for CPU-bound and IO-bound work (`tokio::task::spawn_blocking` and `rayon`).
- **The WebView2 renderer** on its own thread, hosting the React bundle.

No separate Node process, no plugin sidecar, no IPC over sockets. Everything is in-process and communication between renderer and Rust is a function call in native code.

Threads and pools

Where	Rust ownership	Typical work
Main thread	Tauri app loop	Window events, menu, plugin dispatch
Tokio worker threads	`tauri::async_runtime` (multi-threaded)	All `#[tauri::command] async fn` bodies
`spawn_blocking` pool	Tokio's built-in blocking pool	Sidecar/embed writes, EXIF read, thumbnails
`rayon` pool	Global rayon pool	Parallel directory walk, batched hashing
Renderer thread	WebView2	React reconciler, layout, paint

Every Tauri command runs on a Tokio worker. A command that does CPU-bound work (encoding a thumbnail, injecting XMP into a 20 MB JPEG) wraps that in `tauri::async_runtime::spawn_blocking(move || …)` so the worker isn't blocked.

SQLite concurrency

The DB is a single `Mutex<Connection>`. Reads and writes serialise through the lock. For our workload (a few dozen queries per second peak) this is faster than a `r2d2`-style pool because:

- SQLite is single-writer anyway (writes queue at the file level).
- Our reads are short (`< 5 ms`); lock contention is negligible.
- No connection setup / teardown per query.

Long-running queries that would otherwise hold the lock (like a full tag-count refresh across 250 k photos) are structured as one transaction that reads and returns — no cursor kept open — so the lock is released quickly.

Async model

- `async fn` at the Tauri command boundary is idiomatic. It lets us `await` a background thumbnail generation or a metadata write without blocking a Tokio worker.
- Inside a command, DB operations are synchronous (they take a `MutexGuard`). This is deliberate: SQLite calls are fast enough to keep on the worker thread, and it removes a whole class of cancellation bugs.
- **Rule of thumb:** if a call could block > 5 ms (`std::fs::File::read_to_end` of a JPEG, `spawn_blocking` it. Otherwise, run it inline.

Cancellation and shutdown

- Tauri commands don't get an explicit cancellation token; they run to completion. Long batch operations are structured as a loop over IDs so partial progress is durable even if the user closes the window.
- On close, Tauri drops the main window; pending `spawn_blocking` tasks are given a grace period to finish. Any in-flight metadata write is atomic (write-to-temp + rename inside the source image), so worst case a `.write-tmp` file is left behind and cleaned up on the next successful write.

Frontend concurrency

- React Query owns all in-flight IPC promises. It handles deduplication (two components asking for `['image', 5]` share the underlying `invoke` call) and stale-while-revalidate refetches on focus.
- Zustand is synchronous. No middleware, no effects — pure UI state.
- No web workers in v1. The renderer stays responsive because it offloads every non-trivial computation to Rust.

IPC boundary

The contract

The renderer and the Rust core communicate through two mechanisms provided by Tauri:

1. **Commands** — request/response, from renderer to Rust. Each is a Rust `async fn` annotated with `#[tauri::command]` and registered in `lib.rs::invoke_handler`.
2. **Events** — one-way, from Rust to renderer, delivered via `AppHandle::emit(name, payload)` and listened for via `@tauri-apps/api/event::listen`.

Everything is (de)serialised with Serde on the Rust side, JSON on the wire, and TypeScript types on the frontend side. See Tauri command reference for the full list.

Naming conventions

- **Commands** use `snake_case` (Rust) mapped to a same-name string on the frontend: `invoke('update_image_metadata', {…})`.
- **Command arguments** are Rust struct fields in `snake_case`, serialised into the JSON payload as-is. The frontend passes them as `camelCase` object keys because we set `#[serde(rename_all = "camelCase")]` on every argument struct.
- **Events** use a `app://` prefix and a resource-style name: `app://image-updated`, `app://images-deleted`, `app://scan`.

Argument struct design

Every command that takes structured data uses a dedicated struct in `src-tauri/src/types.rs`. Optional fields are `Option<T>`, and any field where "unset" and "set to null" must be distinguished uses the custom `double_option` deserializer — an `Option<Option<T>>` where:

Wire form	Deserialised as	Semantics
`{ }` (omitted)	`None`	Don't touch this field.

Wire form	Deserialised as	Semantics
`{ "field": null }`	`Some(None)`	Clear this field explicitly.
`{ "field": "x" }`	`Some(Some("x"))`	Set to `"x"`.

This distinction matters for the `MetadataPatch` struct: the frontend needs to be able to *clear* a title without also unsetting every other field in the patch.

Type mirroring

`src/types.ts` mirrors the Rust structs. Both sides intentionally avoid derived types (`Omit<T, K>`, `Pick<T, K>`) so a schema change is a single edit in each file.

The frontend types file is a plain declaration file:

```
export interface ImageDetails {
  id: number             // packed global ID
  folderId: number
  path: string           // absolute (folder root + rel_path)
  filename: string
  // ...
  userTags: string[]     // typed by the user inside Magpie
  autoTags: string[]     // read from the file at scan time
  title: string | null
  importedAt: number     // when the row was first inserted
  technical: Array<[string, string]>
  formatHandler: string
}
```

Every `getImage`, `updateImageMetadata`, etc. wrapper in `ipc.ts` declares its argument and return types, so a Rust command that grows a new field lights up a red squiggle in every caller until the frontend catches up.

Error handling

- Rust commands return `AppResult<T>` (an alias for `Result<T, AppError>`).
- `AppError` is a `thiserror::Error` enum with a `Display` that's safe to show to the user (no internal paths in error messages, etc.).
- On the wire, errors serialise to a plain string. The frontend surfaces them via TanStack Query's `onError` callback and console logs.

Events (Rust → renderer)

Event	Payload	Fired when
`app://scan`	`ScanProgress { folder_id, done, total, current }`	During a folder scan.
`app://image-updated`	`i64` (packed global ID)	After successful metadata patch.
`app://images-deleted`	`Vec<i64>` (deleted ids)	After successful delete.

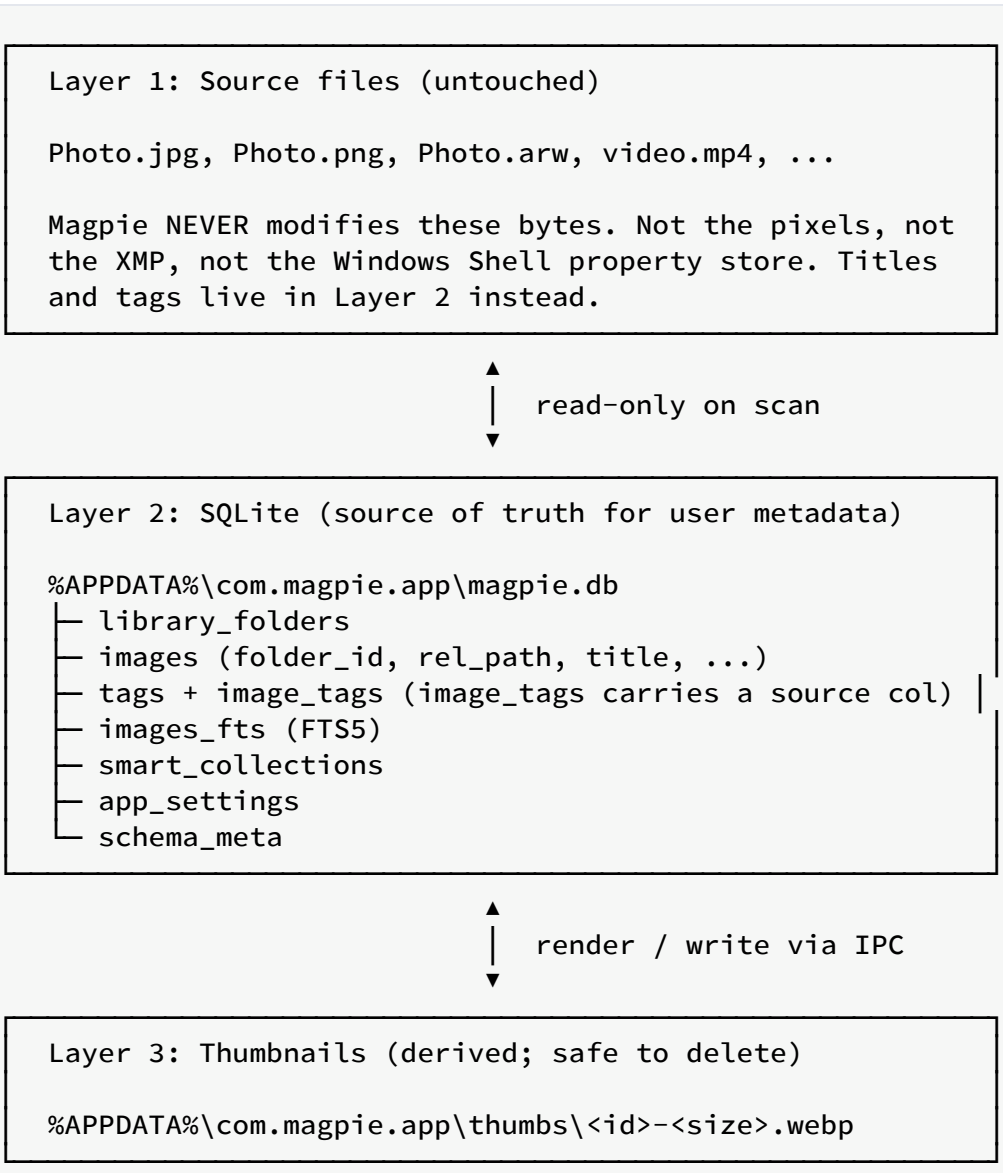
Listeners are attached in `App.tsx` via `useEffect` + `onScanProgress` etc., and each listener updates the appropriate TanStack Query cache or triggers a targeted invalidation.

Capabilities and security

- The renderer has **no direct FS access**. Its `asset:` protocol (used to render thumbnails and full images) is scoped to specific folders declared in `capabilities/default.json`.
- The `tauri.conf.json` `security.csp` blocks inline scripts, remote origins, and `eval`.
- No `dangerouslyUseHttpScheme` or other loosening options are set.
- Commands that operate on paths validate them against the library roots before touching the filesystem.

Data storage architecture

Three layers



Anything a user considers “their tag data” lives in Layer 2. Layer 3 exists purely for speed and can be regenerated at any time by rescanning the library folders.

See Database design for the deeper rationale.

Layer 1: source files (read-only)

Magpie never modifies source files. The scanner reads:

- File metadata (size, mtime) via `std::fs::metadata`.
- Format-specific *technical* metadata (dimensions, EXIF, camera, duration, page count) via each `FormatHandler::read_technical`.
- Format-specific *user* metadata (title, tags) via `FormatHandler::read_user`, and additionally the Windows Shell property store for formats that don't natively parse (RAW, HEIC, MP4, PDF, …).

These reads happen exactly once per file at the first scan (and again if the file's mtime moves forward), then the DB is authoritative.

Existing sidecars and embedded XMP

If the folder already contains XMP-tagged JPEGs (from Lightroom or older Magpie), those tags are imported into `magpie.db` at first-scan time. Older Magpie versions used `.xmp` sidecar files; `core::metadata::sidecar` still reads those on import for backward compatibility, but no new sidecars are ever written.

Layer 2: single central SQLite

One file: `%APPDATA%\com.magpie.app\magpie.db`.

Key properties:

- **Single writer** via a `Mutex<Connection>` inside `Db`. Held only for the duration of a single query or a short transaction.
- **WAL mode** so readers (search, thumbnails, DetailsPanel loads, external DB Browser sessions) never block the writer.
- **Foreign keys with cascade** — deleting a folder wipes its images, which wipes their `image_tags` and FTS rows.
- **FTS5 in `contentless_delete=1` mode** — required for our rebuild-per-row strategy on metadata edits.
- **Plain autoincrement image IDs** — `images.id` is globally unique because there's only one DB. The IPC layer forwards the ID verbatim to the frontend.

Full schema in Database schema.

Layer 3: thumbnail cache

Location: `%APPDATA%\com.magpie.app\thumbs\`.

Format: WebP with quality 80, two sizes per image (`<id>-small.webp`, `<id>-medium.webp`). Small is ~256 px on the long edge, medium ~512 px. `<id>` is `images.id`, which is unique across the whole app.

Regeneration:

- On scan, if a thumbnail is missing or the source file's `mtime` is newer than the thumbnail's, Magpie regenerates.
- On delete, `delete_thumbnails(cache_dir, id)` removes every size.
- The user can safely delete the whole `thumbs\` folder; Magpie regenerates on next request.

Volume assumptions

- Library folders can live on any local volume, including NTFS with long paths (`\\?\`-prefixed).
- Cloud-synced folders (OneDrive, Dropbox, Google Drive, iCloud) and UNC network shares are fine as library roots: the DB itself is in `%APPDATA%` and isn't affected by whatever the sync client does to the photos.
- The central DB and thumbnail cache always live under `%APPDATA%`.

Backup story

`magpie.db` is a single file. Back it up (or the whole `%APPDATA%\com.magpie.app\` directory to include thumbs) and you've backed up every tag, title, smart collection, and setting Magpie knows about.

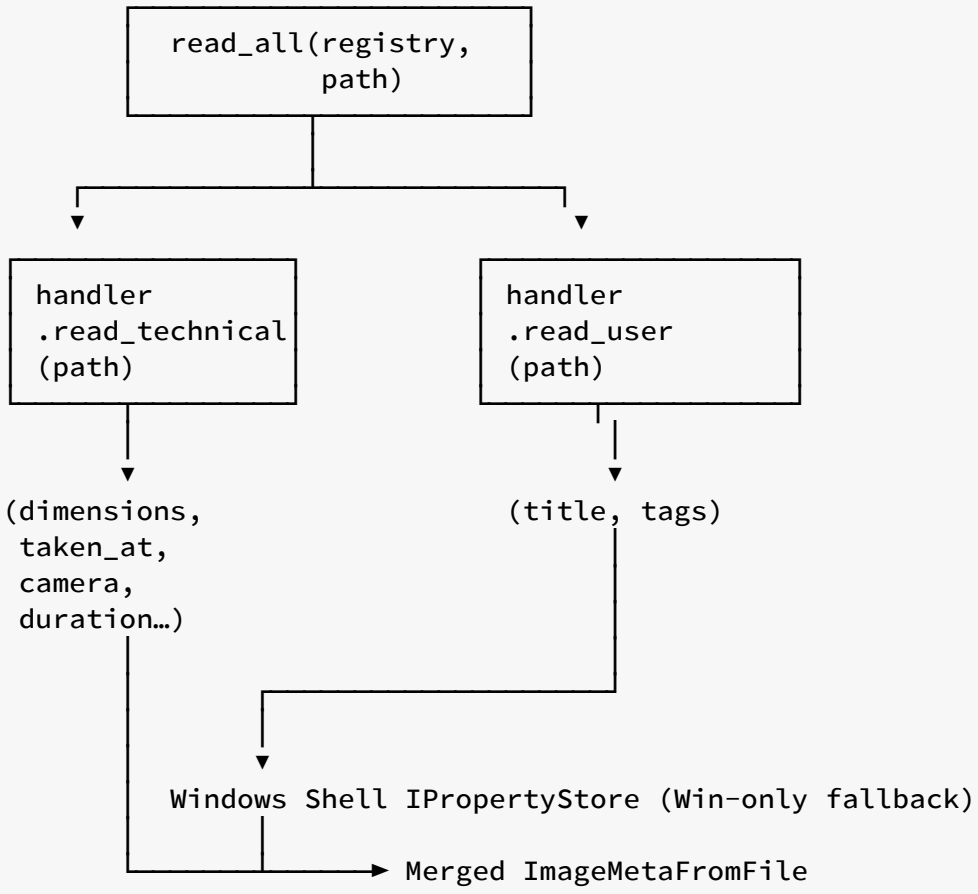
The photo folders themselves don't carry tag data — copying a folder to another disk copies only the pixels. To bring tags along you either copy `magpie.db` too, or re-add the folder on the target PC and let the first scan import whatever the files already embed.

Metadata pipeline

The metadata pipeline is where Magpie most differs from a plain file browser. It has two sub-pipelines (read + patch) and one hard invariant that ties them together:

> **Invariant.** For every file the scanner has seen, the row in `magpie.db` is the single source of truth for user metadata (title + tags). Magpie **never writes back into the source file**. Tags stored in `image_tags` carry a `source` — `'auto'` for the ones Magpie read out of XMP / Windows Shell / sidecars, `'user'` for the ones typed inside Magpie's DetailsPanel. The two are managed independently; see Schema › image_tags.

The read pipeline



Format-native `read_user` runs first; then any Windows-specific Shell property read runs to catch formats we don't natively parse (RAW, HEIC, MP4, PDF, …). Legacy `.xmp` sidecars are also

read at this stage for backward compatibility with older Magpie versions and Lightroom projects.

The XMP parser (`quick_xml` state machine in `core/formats/xmp_packet.rs`) handles both the Adobe standard fields and Microsoft-Explorer variants:

- `dc:title`, `dc:subject` (Alt / Bag containers)
- `MicrosoftPhoto:LastKeywordXMP` (Windows Explorer's tag store)
- Attribute-only forms (some tools flatten `Alt` into an attribute)

`read_all` is called by the scanner on first sight of a file, and by `get_image` if the file's `mtime` has moved forward since last import.

The patch pipeline

Frontend produces a `MetadataPatch` — a struct where every field is `Option`-wrapped so the caller can express "don't touch" vs. "clear" vs. "set":

```
pub struct MetadataPatch {
    pub title: Option<Option<String>>,
    pub tags: Option<Vec<String>>,      // whole list, replaces
    pub tags_add: Option<Vec<String>>,  // additive
    pub tags_remove: Option<Vec<String>>,
}
```

`queries::apply_metadata_patch` runs the whole patch in a single transaction on `magpie.db`. **All tag operations target the `'user'` source**; auto rows are never inserted, removed, or renamed here.

1. Update the `images` row title.
2. If `tags` is set: replace the row's **user** tags (auto rows untouched).
3. If `tags_add` is set: insert missing **user** rows.
4. If `tags_remove` is set: delete matching **user** rows.
5. Rebuild the FTS5 row (DELETE + INSERT, `SELECT DISTINCT` over both sources so a name present in both isn't indexed twice).
6. Commit.

If any step fails, the transaction rolls back. Nothing partially lands.

The batch command (`batch_update_metadata`) loops over each ID and applies the patch inside its own transaction. A failure on one image doesn't roll back the others; the command reports which IDs succeeded.

No write-back to source files

There is deliberately no "write pipeline" any more. `FormatHandler` has no `write_user` method; `win_shell::write_user_meta` is gone; so is the atomic-write helper and every `build_xmp_packet` call site. See Database design.

FS refresh on read

Every `get_image` call does:

```
if fs_meta.mtime > row.mtime_ms {
    let fresh = read_all(&registry, &path);
    queries::set_image_meta(&mut conn, id, &fresh);
}
```

Simple mtime comparison: if the source file changed after Magpie's last import, re-read its metadata into the DB via `queries::set_image_meta`. That path is **additive-only** for tags: each name the file currently reports is inserted as `'auto'` unless the image already carries it in either source; nothing is ever deleted. As a result:

- A tag added in Windows Explorer / Lightroom between scans shows up as a new automatic tag on next click of the file.
- A tag **removed** from the file between scans stays in the DB (still visible under Automatic tags in the details panel) — we can't tell the difference between "file lost the tag" and "file's own metadata is stale", and the safer default is to keep it.
- Tags the user typed inside Magpie are always preserved across rescans, regardless of what the file's own metadata says.

Windows Shell property store (import-only)

For formats without a native XMP parser (RAW, HEIC, MP4, PDF, …), Magpie falls back to Windows' Shell property store to read tags and titles on first scan. `System.Title`, `System.Keywords`, and similar canonical properties are consulted. This ensures that a user who had been tagging RAWs through Explorer's *Properties* → *Details* dialog doesn't lose those tags on migration.

`win_shell` was previously bidirectional; after the redesign it is strictly read-only.

Observability

File-based logging

Every run of Magpie produces a rolling log file at:

```
%APPDATA%\com.magpie.app\logs\app.log
```

The logger is initialised in `src-tauri/src/lib.rs::init_logging`, using `tracing-subscriber` with a plain-text formatter and a `RUST_LOG`-compatible env filter (default: `info,desktop_lib=info`).

Format of a typical line:

```
2026-08-17T18:53:12.331332Z  INFO desktop_lib: Magpie started app_data_dir="…"
2026-08-17T19:22:14.029142Z  INFO desktop_lib::commands::images: get_image:
resynced user metadata from FS id=5
2026-08-18T08:41:03.912483Z  INFO desktop_lib::commands::images:
batch_update_metadata count=3 patch=…
```

- ISO-8601 timestamp with microsecond precision.
- Log level: `TRACE / DEBUG / INFO / WARN / ERROR`.
- Target: usually the module path (`desktop_lib::commands::images`).
- Structured fields on the tail: `id=5`, `count=3`, `?patch`.

Structured fields

We use `tracing`'s field syntax pervasively:

```
tracing::info!(
    id,
    ?patch,                          // Debug-formatted
    op = "update_image_metadata", // static label
    "metadata embedded in source file"
);
```

The `?` prefix invokes `Debug`. Named fields keep the log grep-friendly: to find every batch save, `rg 'batch_update_metadata'` returns just the events, and the id/count/count-succeeded

triplet is on the same line.

Frontend crumbs into the same log

The renderer can push log lines into the Rust log via the `log_frontend` command (`src-tauri/src/commands/diag.rs`). The frontend helper `logFrontend(level, msg)` in `src/ipc.ts` is a best-effort fire-and-forget:

```
logFrontend('info', `applyTags dispatch: ids=${ids.length} add=[…]`)
```

Renderer-side events land in the log with a `target="frontend"` marker, which makes them easy to filter:

```
2026-08-18T08:41:03.900001Z  INFO frontend: applyTags dispatch: ids=3 add=
[vacation] remove=[]
```

This lets us reason about the full IPC round-trip from a single tail of `app.log`.

What's logged where

Signal	Level	Emitted by
App started / logging initialised	INFO	`lib.rs::init_logging`, `lib.rs` main run
Legacy layout migrated into magpie.db	INFO	`db/migrate.rs::open_or_migrate`
Folder registered	INFO	`commands/library.rs::add_library_folder`
Folder root unreachable (drive unplugged)	WARN	`commands/library.rs::list_library_folders`
Command entry (`update_image_metadata`, …)	INFO	Each Tauri command that mutates state
`get_image` FS-refresh triggered	INFO	`commands/images.rs::get_image`

Signal	Level	Emitted by
Scan errors (unreadable file, permission)	WARN	`core/scanner.rs`
DB errors (mutex poisoned, statement failed)	ERROR	`db/mod.rs` , `db/queries.rs` , `lib.rs::run`
Frontend applyTags dispatch	INFO	`features/DetailsPanel.tsx::MultiDetails.applyTa`

Log rotation

The current implementation is a plain append-only file — no rotation. For a v1 release this is fine; the log grows a few kilobytes per session at INFO. A future PR should add `tracing-appender::rolling` with daily rotation or size-based (e.g. 10 MB × 5).

Turning up the volume

Set the `RUST_LOG` environment variable before launching:

```
$env:RUST_LOG = "debug,desktop_lib=trace"
& "$env:LOCALAPPDATA\Programs\Magpie\desktop.exe"
```

This will produce a torrent of scanner-level detail, useful when debugging a bad scan on a specific folder.

No telemetry

There is no automatic upload of the log, no crash reporter, no analytics. `app.log` is on your disk and stays there unless you explicitly share it (e.g. attach it to a bug report).

Repository layout

```
Magpie/
├─ src/                                # React frontend (TypeScript)
│   ├─ App.tsx                         # Root component; layout + event listeners
│   ├─ main.tsx                        # ReactDOM entry
│   ├─ index.css                       # Tailwind directives + global styles
│   ├─ ipc.ts                          # Typed Tauri command wrappers
│   ├─ store.ts                        # Zustand global UI state
│   ├─ types.ts                        # Mirrors Rust types
│   └─ features/
│      ├─ TopBar.tsx                   # Top-of-window: add folder, rescan,
SearchBox, sort
│      ├─ Sidebar.tsx                  # Left: library / folder / tag multi-select
│      ├─ SearchBox.tsx                # Chips (tags) + free-text FTS input
│      ├─ ImageGrid.tsx                # Virtualised file grid (double-click →
Magnifier window)
│      ├─ DetailsPanel.tsx             # Right: SingleDetails / MultiDetails
(Title / Tags / Format / Info + editable filename)
│      ├─ MagnifierWindow.tsx          # Root component of the separate Magnifier
pop-up window
│      ├─ openMagnifierWindow.ts       # Helper that spawns/focuses that window
│      ├─ WelcomeScreen.tsx            # Shown when no project is open
│      ├─ SettingsDialogs.tsx          # Theme / Font-size / Language modals
│      ├─ TagInput.tsx                 # Tag entry with autocompletion
│      ├─ Thumbnail.tsx                # <img> wrapper with async src
│      └─ StatusBar.tsx                # Bottom: scan progress, app version
│
├─ src-tauri/                          # Rust backend (Cargo package)
│   ├─ Cargo.toml
│   ├─ tauri.conf.json                 # App config: identifier, security, window
│   ├─ build.rs
│   ├─ capabilities/
│   │  └─ default.json                 # Capability manifest (asset scopes, etc.)
│   ├─ examples/
│   │  ├─ dump_meta.rs                 # Diagnostic: dump everything about a photo
│   │  └─ dump_tag_usage.rs            # Diagnostic: find photos tagged X
│   ├─ tests/
│   │  └─ metadata_fs.rs               # Integration tests for metadata pipeline
│   └─ src/
│      ├─ main.rs                     # Windows subsystem entry — calls lib::run
│      ├─ lib.rs                      # Tauri builder, logging, handler
registration
│      ├─ menu.rs                     # Native menu bar + menu-event bridge
│      ├─ error.rs                    # AppError, AppResult
│      ├─ types.rs                    # Rust-side IPC types (mirror src/types.ts)
│      ├─ db/
│      │  ├─ mod.rs                   # Db handle (Arc<Mutex<Connection>>)
│      │  ├─ schema.rs                # Apply schema.sql on fresh DBs
│      │  ├─ schema.sql               # DDL for a project DB
│      │  ├─ queries.rs               # All SQL against the project DB
│      │  └─ migrate.rs               # One-shot import from legacy DB layouts
│      │
```

```
│      ├─ core/
│      │   ├─ mod.rs                    # AppServices (holds ProjectState +
AppSettings)
│      │   ├─ project.rs                # Projects + AppSettings persistence
│      │   ├─ scanner.rs                # Parallel folder scan (writes rel_paths)
│      │   ├─ thumbnail.rs              # Thumb generation + caching (keyed by
image_id)
│      │   ├─ formats/                  # Per-format handlers, all read-only
│      │   │   ├─ mod.rs                # FormatHandler trait + FormatRegistry
│      │   │   ├─ common.rs             # EXIF/dims utilities, verbatim-path
helpers
│      │   │   ├─ xmp_packet.rs         # XMP parser (read only)
│      │   │   ├─ win_shell.rs          # Windows IPropertyStore reader
│      │   │   ├─ jpeg.rs
│      │   │   ├─ png.rs
│      │   │   ├─ webp.rs
│      │   │   ├─ gif.rs
│      │   │   ├─ tiff.rs
│      │   │   └─ stubs.rs              # HEIC, PDF, video, RAW, …
│      │   └─ metadata/
│      │       ├─ mod.rs
│      │       ├─ read.rs               # Reads at scan time; populates magpie.db
│      │       └─ sidecar.rs            # Legacy `.xmp` reader (never writes)
│      └─ commands/                     # Tauri command handlers
│          ├─ mod.rs
│          ├─ project.rs                # current/create/open/save[_as]/close
│          ├─ settings.rs               # get/update app settings JSON
│          ├─ library.rs                # add/remove/list folders, rescan
│          ├─ images.rs                 # query, get, update, batch_update, delete,
rename
│          ├─ tags.rs                   # list / rename / delete tags
│          ├─ magnifier.rs              # Magnifier window context (get/set)
│          ├─ collections.rs            # smart collections (skeleton in v1)
│          ├─ thumbs.rs                 # thumb + asset path resolvers
│          └─ diag.rs                   # log_frontend bridge
│
├─ scripts/
│   ├─ screenshot-app.ps1               # Verify UI screenshot
│   ├─ screenshot-scrolled.ps1          # Screenshot with programmatic scroll
│   └─ test-multiselect-tag.ps1         # E2E UI test skeleton
│
├─ docs/                                # This documentation
│   ├─ user-manual/                     # mdBook: user-facing manual
│   ├─ developer/                       # mdBook: this guide
│   ├─ shared/                          # Shared CSS / JS for both books
│   ├─ index.html                       # Landing page linking both docs
│   └─ build.ps1                        # HTML + PDF build script
│
├─ index.html                           # Vite entry
├─ vite.config.ts
├─ tailwind.config.js
├─ postcss.config.js
├─ tsconfig.json
├─ package.json
```

```
├─ README.md
└─ .gitignore
```

What lives where

- **Anything user-visible** (button text, layout, animations) is under `src/`.
- **Anything touching disk** (files, DB) is under `src-tauri/src/`. If you find frontend code doing FS work, that's a bug.
- **Anything that runs at build time** (Tauri config, capabilities, Cargo settings) is under `src-tauri/` outside `src/`.
- **Documentation source** is under `docs/`. Generated HTML lands in `docs/user-manual/book/` and `docs/developer/book/`; PDFs land in `docs/`.

Backend modules

`lib.rs` — app entry

Responsibilities:

- Initialise file-based logging (`init_logging`).
- Build `AppServices { project, settings, thumb_cache_dir, app_data_dir, formats }` under `Arc` .
- Register every Tauri plugin (`dialog` , `opener`) and command handler in `invoke_handler!` .
- Build the native menu via `menu::build_menu` and forward every click to the frontend as an `app://menu` event.
- Manage the app run loop.

Notable pitfalls:

- `init_logging` **must not** call `tracing_subscriber::fmt().init()` if a Tauri plugin (like the removed `tauri-plugin-log`) already set a global subscriber. The prior implementation crashed with `PluginInitialization("log", "attempted to set a logger after…")` ; the current version owns the logger outright.

`core/mod.rs` — `AppServices`

`AppServices` is the shared state every command touches:

```
pub struct AppServices {
    project: Mutex<ProjectState>,
    settings: Mutex<AppSettings>,
    magnifier: Mutex<MagnifierContext>,
    pub app_data_dir: PathBuf,
    thumbs_root: PathBuf,
    pub formats: Arc<FormatRegistry>,
}
```

Key methods:

- `db() -> AppResult<Db>` — return a clone of the currently-open project's `Db` handle. Fails with `AppError::NoProjectOpen` when no project is open. **Every existing command routes SQL through this method**, so a "no project" state is a first-class error instead of a null pointer.
- `current_project()` / `set_project()` — swap the open project; the second also updates `AppSettings.last_project_path` and the recent-projects list, and persists both to disk.
- `get_settings()` / `update_settings(f)` — read or mutate the persisted `AppSettings` blob.
- `thumb_cache_dir()` — returns the **per-project** subdirectory under `thumbs_root` (see Thumbnail pipeline).
- `magnifier_context()` / `set_magnifier_context()` / `set_magnifier_current()` — small piece of state stashed by the main window before spawning the Magnifier pop-up so the pop-up can pull its list context via one command instead of stuffing a serialised `ImageFilter` / `ImageSort` into a URL. See Magnifier window.

`core/project.rs` — project abstraction

A **project** = a single `.magpie` SQLite file the user chose the location of.

Key types:

- `AppSettings` — persisted JSON blob at `%APPDATA%\com.magpie.app\app-settings.json`. Holds theme, font size, language, last-project path, and recent-projects list (bounded to `RECENT_PROJECTS_MAX = 10`).
- `Theme` (`system|dark|light`), `FontSize` (`small|medium|large`).
- `ProjectState { db: Option<Db>, info: Option<ProjectInfo> }` — live state guarded by the `project` mutex on `AppServices`.
- `ProjectInfo { path, name }` — DTO handed to the frontend.

Key functions:

- `create_project(path)` — refuses to overwrite existing files.
- `open_project(path)` — opens an existing DB.
- `save_project_as(current, new_path)` — uses SQLite's online `rusqlite::backup::Backup` API to copy the DB, then reopens.
- `auto_open_on_startup(app_data_dir, &mut settings)` — resolves which project (if any) to open on launch. Priority:
  1. **CLI arg** — if `argv[1]` is an existing `.magpie` file (Windows "open with" hands us the path), open that. Also updates `last_project_path`.
  2. `settings.last_project_path` if it points at an openable file.

3. Legacy `%APPDATA%\com.magpie.app\magpie.db` — renamed in place to `Default.magpie` and **added to** `recent_projects` but **not** auto-opened. The user still sees the welcome screen on their first launch after upgrading and can click the migrated project to open it. This deliberately gives up "silently resume" for one-off migration launches so requirement 5.1 (buttons visible when no project is open) always holds.
4. Nothing.

`menu.rs` — native menu bar

Builds the `Project / Edit / View / Settings` menu via Tauri 2's `MenuBuilder`. Each item has a stable string ID (`ID_PROJECT_NEW`, `ID_EDIT_UNDO`, …). Clicks are forwarded to the renderer as `app://menu` events; the frontend's `useMenuRouter` decides what to do.

The `set_menu_item_enabled(id, enabled)` command lets the frontend grey out context-sensitive items. Currently used for:

- `edit_undo` / `edit_redo` — driven by undo/redo stack length.
- `view_magnifier` — driven by `useStore.selection.primary`; the menu item ships disabled and is only enabled when the user selects a file. Matches the requirement to disable "View → Magnifier" when nothing is selected.

Magnifier window

The magnifier is a **separate native** `WebviewWindow` (label `"magnifier"`), created from the frontend via `new WebviewWindow(…)` when the user double-clicks a tile or picks View → Magnifier. It loads the same bundle as the main window with `url: "index.html#magnifier"`; `src/main.tsx` inspects `location.hash` and renders `<MagnifierWindow />` instead of `<App />` for that route.

Coordination between the two windows lives in `AppServices`:

```
pub struct MagnifierContext {
    pub image_id: Option<i64>,
    pub filter: ImageFilter,
    pub sort: ImageSort,
}
```

Flow:

1. Main window calls `set_magnifier_context(image_id, filter, sort)` (`commands/magnifier.rs`) before creating the window. This stashes the DTO in the mutex.
2. If a magnifier window already exists it's focused and receives an `app://magnifier-reset` event so it re-reads the context and jumps to the new image.
3. Otherwise `new WebviewWindow` creates it. The popup calls `get_magnifier_context()` on mount, queries the same filtered + sorted set as the grid, and displays the current image.
4. Arrow-key navigation calls `set_magnifier_current(image_id)` so the "last shown" pointer stays fresh.

The magnifier window is scoped in `capabilities/default.json`:

```
"windows": ["main", "magnifier"],
"permissions": [
  "core:webview:allow-create-webview-window",
  "core:window:allow-close",
  "core:window:allow-set-focus",
  "core:window:allow-set-title",
  ...
]
```

Both windows are on the same origin so IPC, the asset protocol, and the shared SQLite handle all Just Work.

`types.rs` — IPC types

- `LibraryFolder` — one registered folder; includes `isAvailable`.
- `ImageSummary`, `ImageDetails` — full and abridged views of an image row. `id` is the plain `images.id` primary key of the open project's DB.
- `MetadataPatch` — the "what to change" payload for `update_image_metadata` and `batch_update_metadata`.
- `double_option` module — custom Serde deserializer for `Option<Option<T>>`. Distinguishes "field missing" from "field explicitly null" on the wire.

`ProjectInfo`, `AppSettings`, and `AppSettingsPatch` live in `core::project` and `commands::settings` respectively (both `Serialize + Deserialize` with camelCase JSON).

Every struct uses `#[serde(rename_all = "camelCase")]` so JSON keys are camelCase but Rust fields stay snake_case.

`error.rs`

`AppError` enum (via `thiserror`):

- `Io(std::io::Error)` — filesystem errors.
- `Db(rusqlite::Error)` — DB errors.
- `Pool(String)` — mutex-poisoning errors on the shared `Db` handle. (Legacy variant name; kept for compatibility.)
- `NoProjectOpen` — every DB-touching command returns this when no project is currently open. The frontend interprets it as "route the user to the welcome screen".
- `MetadataRead(String)` — user-facing metadata read failures.
- `Internal(String)` — anything else.

All commands return `AppResult<T>` (= `Result<T, AppError>`). The `Display` impl for `AppError` is safe to surface to the user (no absolute paths in strings without context).

db/ — single-project storage

See Database schema and Database design.

- `db/mod.rs` — the `Db` handle wrapping `Arc<Mutex<Connection>>` plus `open`, `with_conn`, `with_conn_mut`. Defines `DB_FILE_NAME = "magpie.db"` (used only as the legacy filename and internal default) and `SCHEMA_VERSION`.

- `db/schema.rs` + `db/schema.sql` — DDL for a fresh DB and the version check for an existing one.

- `db/queries.rs` — every query in one file, grouped by concern: folders, images (upsert / meta / delete / **rename**), MetadataPatch, tag rename/delete, search (`query_images`, `list_all_tags`), smart collections. Tag operations are **source-aware**:

  - `set_image_meta` (scanner path) inserts `'auto'` rows only when the name isn't already carried by the image (never deletes).
  - `add_auto_tags_for_image` (automatic AI tagging path, `core::auto_tag`) uses the same additive-only rule, so AI suggestions merge with XMP-derived auto tags in a single read-only bucket.
  - `apply_metadata_patch` (UI path from the details panel) only ever touches `'user'` rows.

  See Schema › image_tags for the split.

- `db/migrate.rs` — startup importer for the two pre-project layouts (per-folder `.magpie/library.db` and the earlier central `library.db`). Runs the first time the app sees them and produces the project's SQLite file.

`core/scanner.rs`

Pipeline for `add_library_folder` and `rescan_*`:

1. **Walk.** `jwalk::WalkDir` traverses the folder in parallel.
2. **Filter.** Extensions checked against `FormatRegistry::all_extensions`. Leftover `.magpie/library.db` (from a failed prior migration) is excluded from the walk.
3. **Diff.** For each file, look up by `(folder_id, rel_path)` in `images`; skip if `mtime_ms` matches.
4. **Extract.** In parallel via a `tokio` semaphore: EXIF read, XMP read (native handlers), Windows Shell property read (formats without native parsers), content hash (`XXH3`).
5. **Upsert.** DB writes are serialised via the `Db` mutex; each write is a short single-statement borrow.
6. **Thumbnails.** Enqueue small + medium keyed by `images.id`.
7. **Progress.** Emit `app://scan { folder_id, processed, total, current }`.

Every DB touch goes through `services.db()?` first, so if the user closed the project mid-scan the current file simply errors out and the scan aborts cleanly.

`core/auto_tag/`

Optional pipeline chained onto the tail of a successful scan when `AppSettings.aiAutoTag` is on:

- `mod.rs` — `tag_folder(services, app_handle, folder_id)`. Same shape as `scanner::scan_folder`: enumerate candidates, run per-image work on a semaphore, emit `app://auto-tag` progress events (payload `AutoTagProgress`). All AI passes go through a single `tokio::sync::Mutex<()>` on `AppServices.auto_tag_gate` so multiple newly-added folders queue up FIFO instead of thrashing.
- `classifier.rs` — the small `ImageClassifier` trait (`classify(bytes) → Vec<TagSuggestion>` + `min_confidence` + `max_tags_per_image`) plus the deterministic `MockClassifier` used exclusively in tests.

- `clip_classifier.rs` — production `ImageClassifier` backed by OpenAI's **CLIP-ViT-B/32**, driven by `candle` (https://github.com/huggingface/candle) (pure-Rust ML; no C++ or ONNX Runtime dependency). Per-image work:

  1. `preprocess_image` — decode with `image::load_from_memory`, resize the short side to 224, centre-crop 224×224, split into 3×224×224 NCHW `[0,1]`, then apply CLIP mean/std normalisation.
  2. `ClipModel::get_image_features` on `Device::Cpu` → 512-dim embedding, L2-normalised.
  3. Cosine-similarity dot-product against the pre-computed text embedding matrix, keep the top-`MAX_TAGS_PER_IMAGE=6` above `MIN_COSINE=0.20`.

  Vocabulary is compiled in from `core/auto_tag/resources/photo_vocab_v1.txt` (~1000 photo words). Text embeddings for that vocab are computed once via `compute_text_embeddings` (BPE tokenise with the CLIP tokenizer, pad to 77 tokens, run `get_text_features`, L2-normalise) and cached under `<app_data_dir>/models/clip/photo_vocab_v1.embeddings.f32` keyed by `SHA-256(photo_vocab_v1.txt)` — bumping the vocab file invalidates the cache automatically.

- `model_manager.rs` — downloads and verifies the two required files under `<app_data_dir>/models/clip/`: `model.safetensors` (~605 MB, pinned SHA-256) and `tokenizer.json` (~2 MB, trusted). Streams bytes with `reqwest` + `futures-util` to a `.part` file, verifies the hash, then atomically renames. Publishes `app://ai-model-download` progress events (payload `AiModelDownloadProgress`) throttled to 200 ms so the dialog can draw a smooth progress bar.

  A few details worth knowing when debugging network failures:

  - `reqwest` is built with the `rustls-tls-native-roots` feature so TLS uses the **Windows trust store** (SChannel roots via `rustls-native-certs`). This is required when the user sits behind a corporate MITM proxy whose CA is not in the bundled Mozilla `webpki-roots` set.
  - Downloads are **resumable**. Bytes are hashed as they land in the `.part` file, and if a request fails mid-stream the next attempt sends `Range: bytes=N-` from wherever the hash counter stopped, so a mid-transfer drop costs one round-trip rather than 600 MB.
  - Each file is retried up to `MAX_DOWNLOAD_ATTEMPTS = 4` times with 2s / 4s / 8s exponential backoff. On a hard failure the error message contains the full `reqwest → hyper → rustls` source chain (`chain()` helper walks `.source()` recursively), which is what the Settings dialog surfaces to the user.
  - Per-request timeouts: 30 s to connect, 30 min for the whole body, plus TCP keep-alive every 30 s so stateful firewalls don't silently drop the connection during long

downloads.

- `tag_folder` refuses to spawn the classifier when `model_manager::check_status` reports the safetensors or tokenizer files missing — it emits a single "finished with error" `AutoTagProgress` so the status bar shows a warning, and the Settings dialog is the only path that starts a download.

Per-image bookkeeping lives on the `images` row itself: `ai_tagged_at` (Unix ms of the last successful pass) and `ai_tag_hash` (the file's fingerprint — `content_hash` when known, else `mtime_ms.to_string()`). `queries::list_auto_tag_candidates` skips rows whose fingerprint still matches, so a rerun on an unchanged folder is O(candidates) DB reads and no classifier work. See Scanner → Auto-tag pass for the full stage-by-stage description.

core/thumbnail.rs

```
pub fn ensure_thumbnails(cache_dir: &Path, src: &Path, image_id: i64) ->
Result<()>;
pub fn thumb_path(cache_dir: &Path, image_id: i64, size: ThumbSize) -> PathBuf;
pub fn delete_thumbnails(cache_dir: &Path, image_id: i64);
```

Decode via `image::open`, resize via `fast_image_resize` (SIMD Lanczos3), encode via `webp::Encoder` at quality 80. `image_id` is the plain `images.id` primary key.

Thumbnail cache is scoped per project. `AppServices` exposes `thumb_cache_dir()` which returns `%APPDATA%\com.magpie.app\thumbs\<key>\` where `<key>` is a 64-bit hash of the currently-open project's absolute path (case-insensitive on Windows). This guarantees two projects that happen to share the same `image_id` — SQLite's autoincrement is per-DB — never see each other's cached previews. `thumb_cache_dir()` returns `AppError::NoProjectOpen` when nothing is open, and every call site handles that gracefully.

The hashing function `project::thumb_cache_key(path)` is covered by unit tests for stability and case handling.

core/metadata/read.rs

`read_all(registry, path) -> ImageMetaFromFile` runs the read pipeline:

1. Handler's `read_technical` for dimensions + EXIF-derived fields.
2. Handler's `read_user` for XMP-parseable formats.

3. Windows Shell property store fallback for RAW / HEIC / MP4 / PDF.
4. Legacy `.xmp` sidecar read for backward compatibility (no writes).

Non-fatal errors are collected into a per-file warning log; the scanner continues on the next file.

The full pipeline is described in Metadata read path.

`core/formats/`

Each supported extension is owned by one `FormatHandler` implementation, and every handler is **read-only** (see Database design):

- `mod.rs` — declares the `FormatHandler` trait (`name`, `extensions`, `kind`, `read_technical`, `read_user`), `TechnicalMeta`, `UserMeta`, `FormatKind`, and the `FormatRegistry` that lives on `AppServices`.
- `xmp_packet.rs` — hand-written streaming XMP parser (no writer). Extracts packets containing title, subjects, description, and Microsoft-Photo keywords.
- `common.rs` — shared helpers: EXIF → technical metadata, dimensions, verbatim-prefix stripping.
- `jpeg.rs`, `png.rs`, `webp.rs`, `gif.rs`, `tiff.rs` — native readers.
- `stubs.rs` — HEIF, video, PDF, RAW, BMP/EXR/HDR/SVG readers. These lean on `imagesize` for dimensions and defer user metadata to `win_shell::read_user_meta`.
- `win_shell.rs` — Windows-only Shell property store **reader**.

No `metadata/write.rs`

The write path is gone. `FormatHandler` has no `write_user`; `common::atomic_write_bytes`, `win_shell::write_user_meta`, `xmp_packet::build_xmp_packet`, and `xmp_packet::merge_user_edits` were all removed. Every user-metadata mutation now goes through `db::queries::apply_metadata_patch` and lives in the project DB.

The single exception is **file renames** via `rename_image` — `std::fs::rename` on disk followed by `queries::rename_image_row` in one command. This is the only place Magpie modifies anything under a source folder, and only when the user explicitly asks.

`commands/*.rs`

Each file exposes 3–5 `#[tauri::command]` async functions. See Tauri command reference for the full list.

Common patterns:

- `services: State<'_, Arc<AppServices>>` for shared state.
- `let db = services.db()?;` at the top of every DB-touching command — surfaces `NoProjectOpen` as a normal error.
- `db.with_conn(...)` / `db.with_conn_mut(...)` for every SQL operation.
- `app_handle: AppHandle` for emitting events.
- Return `AppResult<T>`; Serde does the JSON legwork.
- Tracing spans on entry so `app.log` shows every IPC hit.

Frontend modules

The frontend is intentionally small — the goal is to be a translation layer between UI events and Tauri commands, not to hold domain logic.

`src/main.tsx`

Boots React. Wraps the rendered component in a `QueryClientProvider` with a single QueryClient configured for:

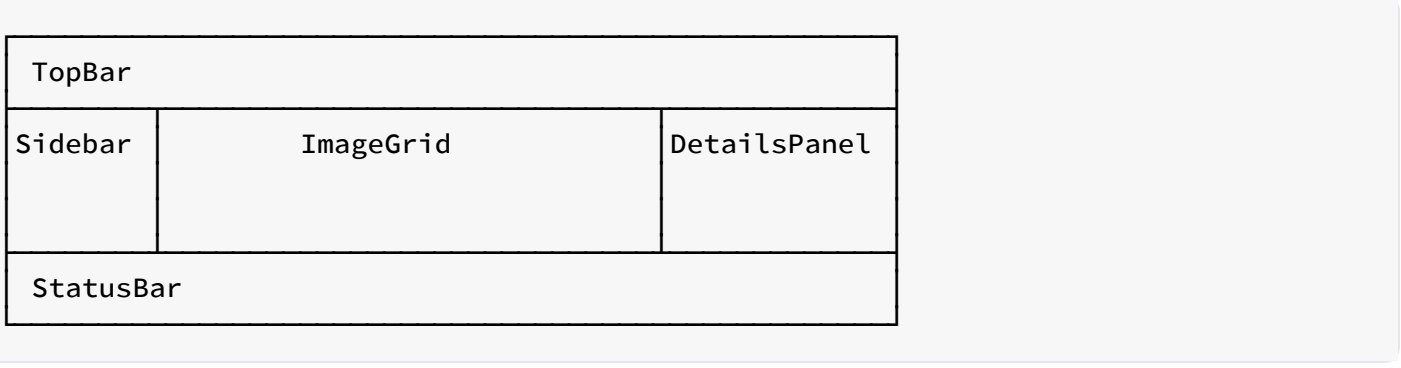
- `staleTime: 0` — refetch on refocus.
- `retry: 1` — one retry on network / IPC errors.
- Query cache keys convention (below).

`main.tsx` also acts as a tiny router: if `location.hash` is `#magnifier` we render `<MagnifierWindow />` (see below), otherwise `<App />`. This lets both windows share one Vite bundle instead of requiring a second HTML entry point.

`src/App.tsx`

Owns the top-level layout, project bootstrap, and menu routing.

The main-UI wrapper `<div>` is `key`ed on `project.path` so switching projects fully unmounts the tree — this resets React Query state, virtualised grid scroll, and the `Thumbnail` `<img src>` caches so nothing from the previous project leaks through when the new one loads.



Project bootstrap:

- Fetches `current_project` and `get_app_settings` on mount via React Query, then seeds `useStore` accordingly.
- If `current_project` returns `null`, renders `<WelcomeScreen />` instead of the layout above.
- Applies `settings.theme` and `settings.fontSize` to the `<html>` element (adds `theme-*` and `font-size-*` classes).

Menu routing:

- `useMenuRouter({ … })` subscribes to `app://menu` and dispatches each menu ID to a handler (`handleNewProject`, `handleOpenProject`, `handleSaveAs`, `handleClose`, `handleOpenMagnifierFromMenu`, `handleUndoOrRedo('undo' | 'redo')`, `setSettingsDialog(…)`).
- Separate `useEffect`s call `setMenuItemEnabled` for `edit_undo` / `edit_redo` (driven by stack length) and `view_magnifier` (driven by `selection.primary != null`).

Also hosts:

- The global keyboard listener (Delete, Escape).
- `app://image-updated` and `app://images-deleted` listeners that invalidate the relevant query keys.
- The `<Magnifier />` and `<SettingsDialogs />` overlay mounts.

`src/store.ts` (Zustand)

Global UI-only state:

```
interface Store {
  // Project + app settings
  project: ProjectInfo | null
  settings: AppSettings | null
  setProject, setSettings, setTheme, setFontSize

  // Browse + search
  view: View                // 'all' | { kind: 'folder', folderId } | 'untagged' |
'missing'
  search: string
  selectedTags: string[]    // AND-combined; drives sidebar tag bubbles +
SearchBox chips
  sort: ImageSort
  extraFilter: ImageFilter
  toggleSelectedTag, addSelectedTag, removeSelectedTag,
    setSelectedTags, clearSelectedTags

  // Selection + right pane
  selection: { ids: Set<number>; anchor: number | null }
  detailsOpen: boolean

  // Session-scoped undo/redo
  undoStack: UndoEntry[]
  redoStack: UndoEntry[]
  pushUndo, popUndo, pushRedo, popRedo, clearHistory
}
```

`setProject(new)` clears the entire per-project session (view, selection, tags, search, undo/redo). This is what keeps the UI honest when the user switches projects. The magnifier lives in a separate window and has its own state on the Rust side, so it does **not** live in this store.

`filterFromView(view, extra, search, selectedTags)` composes the sidebar view, the extra filter, the free-text search, **and the selected-tag list** into one `ImageFilter` sent to `query_images`. Multiple tags AND together server-side.

`UndoEntry`:

```
type UndoEntry =
  | { kind: 'title';  id: number; from: string; to: string }
  | { kind: 'tags';   id: number; from: string[]; to: string[] }
  | { kind: 'rename'; id: number; from: string; to: string }
```

Undo/redo is session-scoped (in-memory, per app run). Closing the project or the app clears it.

src/ipc.ts

Thin, typed wrappers around `invoke`:

```
export const updateImageMetadata = (id: number, patch: MetadataPatch) =>
  invoke<ImageDetails>('update_image_metadata', { id, patch })

export const renameImage = (id: number, newFilename: string) =>
  invoke<ImageDetails>('rename_image', { id, newFilename })

export const currentProject = () =>
  invoke<ProjectInfo | null>('current_project')

export const createProject = (path: string) =>
  invoke<ProjectInfo>('create_project', { path })

// …plus openProject, saveProject, saveProjectAs, closeProject,
//    getAppSettings, updateAppSettings, setMenuItemEnabled.
```

Event-listener helpers:

```
export const onImageUpdated = (h: (id: number) => void) =>
  listen<number>('app://image-updated', e => h(e.payload))

export const onMenuEvent = (h: (menuId: string) => void) =>
  listen<string>('app://menu', e => h(e.payload))
```

And the diagnostic helper `logFrontend(level, msg)` for pushing crumbs into the backend log.

src/types.ts

Every IPC struct duplicated from Rust, kept in sync manually. See IPC boundary for the rationale (no derived types).

Notable additions for the project model:

```ts
export type ProjectInfo = { path: string; name: string }
export type Theme      = 'system' | 'dark' | 'light'
export type FontSize   = 'small' | 'medium' | 'large'
export type AppSettings = {
  theme: Theme
  fontSize: FontSize
  language: string
  lastProjectPath: string | null
  recentProjects: string[]
}
export type AppSettingsPatch =
  Partial<Pick<AppSettings, 'theme' | 'fontSize' | 'language'>>
```

src/features/

TopBar.tsx

- Add-folder button (opens Tauri dialog, calls `add_library_folder`, invalidates `['folders']` and `['images']`).
- Rescan button (calls `rescan_all`).
- `<SearchBox />` — chips for every selected tag plus free-text search input. Not present on the welcome screen.
- Sort dropdown + direction toggle.
- Toggle-details icon.
- Current project name (with full path on hover) at the right edge.

SearchBox.tsx

- Renders one `<Chip>` per entry in `useStore.selectedTags`; clicking the chip's `×` calls `removeSelectedTag`, which automatically deselects the sidebar tag bubble for that tag.
- Free-text input is debounced (200 ms) into `useStore.setSearch`.
- Enter submits the current draft.

Sidebar.tsx

- `['folders']`, `['tags']` queries fed to collapsible sections.
- Folders/quick-filters set `useStore.view` and clear the selection.

- Tags render as `<TagBubble>` pills inside a `flex flex-wrap` container, so they flow across as many rows as they need in the sidebar's width. Clicking a bubble calls `toggleSelectedTag`; selected bubbles are filled with the accent colour and unselected bubbles use the raised-surface background. Each bubble carries a small numeric badge with the tag's usage count. A `Clear all` button at the top of the Tags section calls `clearSelectedTags`.
- Right-click context menu for `Remove folder`, `Rescan folder`, `Rename tag`, `Delete tag`.

`ImageGrid.tsx`

- `useQuery(['images', filter, sort, page])` returns paginated results. `filter` is produced via `filterFromView(view, extra, search, selectedTags)`.
- `useVirtualizer` from TanStack Virtual computes visible rows.
- Each tile is a `Thumbnail` component; `Ctrl / Shift / plain` click logic mutates `useStore.selection`.
- **Double-click** on a tile calls `useStore.openMagnifier(img.id)`.

`DetailsPanel.tsx`

- Reads `useStore.selection` and dispatches:
  - 0 selected → placeholder.
  - 1 selected → `<SingleDetails id=… />`.
  - 1 selected → `<MultiDetails ids=… />`.

`SingleDetails` renders the following sections plus an editable file name:

1. **Title** — editable input, auto-saves via `updateImageMetadata`. Pushes an `UndoEntry` for every successful save.
2. **Your tags** — `TagInput`, backed by `ImageDetails.userTags`. Auto-saves via `updateImageMetadata` (which targets the `'user'` source on the Rust side). Also pushes an `UndoEntry`.
3. **Automatic tags** — read-only `<ReadOnlyTagList>` fed by `ImageDetails.autoTags`. **Always rendered**, even when the list is empty (an italic "No automatic tags on this file." placeholder takes the pill row's place), so the distinction between editable and read-only tags stays discoverable on every file. Extra affordances make the read-only nature obvious:

    - A `<LockIcon>` next to the section label.
    - Each pill carries its own small lock glyph.
    - The container uses a dashed border, muted colours, and `cursor-not-allowed`.
    - `aria-readonly="true"` on the container so screen readers announce it correctly. The mirror section **Your tags** carries a `<PencilIcon>` in the same slot so the two categories are visually paired.
4. **Format metadata** — read-only: handler name. Room to grow into per-format editable fields (GPS, description, …) as those handlers grow their surface area.
5. **File info** — `<dl>` of the `technical` list returned by the backend plus filename, size, format, mtime, and import time. The **filename** is inline-editable: Enter calls `renameImage`, Escape or blur reverts the input.
6. **Preview** — double-clicking the small preview opens the Magnifier for that image.

Local edit state is seeded from the query result only when the `id` changes (guarded by `lastLoadedId.current`), avoiding the "stomp typing on refetch" bug.

`MultiDetails` shows two `TagInput`s (Add / Remove) and an Apply button. It uses refs (`tagsAddRef`, `tagsRemoveRef`) mirroring state so the mutation dispatch always sees the latest values, even if a blur-triggered `setTagsAdd` and a click on `Apply` land in the same React batch.

`MagnifierWindow.tsx` + `openMagnifierWindow.ts`

The magnifier is a **separate native Tauri window**, not an in-app modal.

- `openMagnifierWindow(imageId, filter, sort)` (helper in `features/openMagnifierWindow.ts`) is called by `ImageGrid.onDoubleClick`, `DetailsPanel`'s preview double-click, and `App.tsx`'s `View → Magnifier` menu handler. It first calls `setMagnifierContext` to stash the DTO on the Rust side, then either focuses the existing `WebviewWindow` labelled `"magnifier"` (and emits `app://magnifier-reset` to refresh it) or creates a new one pointing at `index.html#magnifier`.
- `MagnifierWindow.tsx` is what `main.tsx` mounts for that route. It fetches `getMagnifierContext()` on mount, runs the same `queryImages(filter, sort, { limit: 5000 })` the grid is using, and paints the current image via `<img src={toAssetUrl(cur.path)}>`. Prev / Next / Esc all work; the window title mirrors the current filename.

Both windows share the same origin (and therefore the same asset protocol scope, IPC handler, and SQLite handle), so nothing extra is needed to render pictures from the source folders.

`WelcomeScreen.tsx`

- Rendered by `App.tsx` when `useStore.project === null`.
- Two primary buttons: **New Project…** (Tauri save dialog) and **Open Project…** (Tauri open dialog).
- List of recent projects from `AppSettings.recentProjects`, each row invokes `openProject`.
- On success calls `setProject(info)` and lets React Query refetch everything for the new project.

`SettingsDialogs.tsx`

- Modal shell with a small radio-group form.
- `<ThemeDialog>` — System / Dark / Light. Updates `settings.theme` via `updateAppSettings`; `App.tsx` reapplies the `<html>` class immediately.
- `<FontSizeDialog>` — Small / Medium / Large. Same pattern.
- `<LanguageDialog>` — placeholder in v1 (English only).

`TagInput.tsx`

- Controlled by parent's `tags: string[]` + `onChange(next: string[])`.
- Commits the current draft on Enter, Space, Comma, or blur.
- Autocompletion pulled from `list_tags(prefix)` via TanStack Query.
- Backspace on empty input pops the last tag.

`Thumbnail.tsx`

- Resolves `getThumbPath(id, 'small')` on mount, sets `<img src>` to the returned asset URL.
- Placeholder while the thumbnail is being generated (rare, only on first scan).

`StatusBar.tsx`

- Listens to `app://scan` events, shows a live progress bar and message.
- Static labels for app version, DB size, thumbnail cache size.

Query key conventions

Query key	Fetched by	Invalidated by
`['project']`	`currentProject`	any `project::*` command
`['settings']`	`getAppSettings`	`updateAppSettings`
`['folders']`	`listLibraryFolders`	add/remove/rescan folder, delete images
`['tags']`	`listTags`	any metadata update, tag rename/delete
`['tag', prefix]`	`listTags(prefix)`	any metadata update
`['images', filter, sort, page]`	`queryImages`	any image mutation
`['image', id]`	`getImage`	`update_image_metadata` (via `setQueryData`),
		`batch_update_metadata` (via `invalidateQueries`),
		`rename_image`,
		`app://image-updated` event
`['smartCollections']`	`listSmartCollections`	create/delete collection

Styling

- Tailwind for utility classes.
- Global styles in `src/index.css`: dark background, rounded buttons, focus rings.
- Theme classes (`html.theme-light`, `html.theme-dark`, `html.theme-system`) apply the light-mode overrides on top of the default dark palette.
- Font-size classes (`html.font-size-small|medium|large`) set the root `font-size` so every `rem`-based Tailwind value scales.
- Modal styles (`.modal-backdrop`, `.modal-panel`, `.modal-card`) are used by `Magnifier` and `SettingsDialogs`.
- No CSS-in-JS. Component-scoped styles are className concatenations via `clsx` when needed.

Database schema

Everything Magpie persists lives in one SQLite file: `magpie.db` in the app-data directory. See Database design for the motivation and the two earlier layouts we've migrated away from.

Location: `%APPDATA%\com.magpie.app\magpie.db` (Windows). **Owner:** `src-tauri/src/db/schema.rs` reads the DDL from `schema.sql` and applies it verbatim on a fresh file. **Pragmas:** WAL journal, `synchronous=NORMAL`, foreign keys on, `busy_timeout=5000`.

Tables

`library_folders`

Registered root folders. Grows with the number of folders the user adds, not with the number of files.

Column	Type	Notes
id	INTEGER	PK, autoincrement.
path	TEXT	Absolute canonical path, UNIQUE COLLATE NOCASE.
added_at	INTEGER	Unix ms.
last_scan_at	INTEGER	Unix ms; NULL before first scan finishes.
is_available	INTEGER	0/1; 0 when the folder root can't be reached.

`images`

One row per file (of any format the registry recognises) scanned into any library folder. Table name is historical; it now holds videos, PDFs, and documents too.

Column	Type	Notes
id	INTEGER	PK, autoincrement. Globally unique.
folder_id	INTEGER	FK → `library_folders(id)` ON DELETE CASCADE.

Column	Type	Notes
rel_path	TEXT	Folder-relative, forward slashes.
filename	TEXT	Basename (`IMG_1234.jpg`).
ext	TEXT	Lowercase, no leading dot.
size_bytes	INTEGER	From `fs::Metadata::len()` .
mtime_ms	INTEGER	Modification time (Unix ms).
width	INTEGER	Nullable; from handler's technical read.
height	INTEGER	Nullable.
content_hash	TEXT	Nullable; XXH3 128-bit hex digest.
taken_at	INTEGER	Nullable; Unix ms from EXIF `DateTimeOriginal` .
camera_make	TEXT	Nullable.
camera_model	TEXT	Nullable.
title	TEXT	Nullable; user-editable.
imported_at	INTEGER	Unix ms when the row was first inserted.
missing	INTEGER	0/1; 1 after a scan that failed to see the file.
ai_tagged_at	INTEGER	Nullable; Unix ms of the last successful AI-tag pass.
ai_tag_hash	TEXT	Nullable; fingerprint (`content_hash` or `mtime_ms`) at the time of that pass; the AI pipeline skips a row on re-run when this still matches.

UNIQUE: `(folder_id, rel_path)` — the scanner upserts on this key. **Indexes:** `idx_images_folder` , `idx_images_taken_at` , `idx_images_filename` , `idx_images_ext` , `idx_images_hash` .

`tags`

Column	Type	Notes
id	INTEGER	PK, autoincrement.

Column	Type	Notes
name	TEXT	UNIQUE COLLATE NOCASE.

Global vocabulary. `"Beach"` and `"beach"` end up in the same row.

`image_tags`

Many-to-many join. Every row records **who attached the tag** via the `source` column:

- `'auto'` — imported from the file's own metadata (XMP subjects, Windows Shell keywords, sidecar XMP) at scan time.
- `'user'` — added by the user inside Magpie via the DetailsPanel.

The same `(image, tag)` pair may exist twice — once from each source — and the sidebar / FTS index treat them as one tag when aggregating.

Column	Type	Notes
image_id	INTEGER	FK → `images(id)` ON DELETE CASCADE.
tag_id	INTEGER	FK → `tags(id)` ON DELETE CASCADE.
source	TEXT	`CHECK (source IN ('auto','user'))`.
PK		`(image_id, tag_id, source)`.

Indexes:

- `idx_image_tags_tag(tag_id)` — vocabulary → images.
- `idx_image_tags_source(image_id, source)` — one image's tags of a given kind, used by `user_tags_for_image` / `auto_tags_for_image` in `db::queries`.

Who writes what:

- The **scanner path** (`db::queries::set_image_meta`, called on scan and on any mtime-triggered re-read) inserts `'auto'` rows only for names the image doesn't already carry (in either source). It never deletes anything, so a user tag survives a rescan even when the file's own metadata no longer mentions it, and an auto tag stays put even if the source file drops it.
- **User edits** (`db::queries::apply_metadata_patch` from the DetailsPanel) only ever touch `'user'` rows. `MetadataPatch.tagsRemove` deletes the matching `'user'` row and leaves any `'auto'` row alone.

`images_fts` (virtual, FTS5)

```
CREATE VIRTUAL TABLE images_fts USING fts5(
    title, filename, tags,
    content='',
    contentless_delete=1,
    tokenize='unicode61 remove_diacritics 2'
);
```

- Contentless (`content=''`) — `rowid` mirrors `images.id`; we join from FTS matches back to the base table.
- `contentless_delete=1` — required for our rebuild-per-row strategy (`DELETE + INSERT` inside every metadata patch).
- Diacritics folded (`naive` = `naïve`).

`smart_collections`

Column	Type	Notes
id	INTEGER	PK, autoincrement.
name	TEXT	
filter	TEXT	JSON-serialised `ImageFilter`.
sort_order	INTEGER	

`app_settings`

Key-value store for app-wide preferences the UI persists across launches.

Column	Type	Notes
key	TEXT	PK.
value	TEXT	Free-form (JSON, plain string, …).

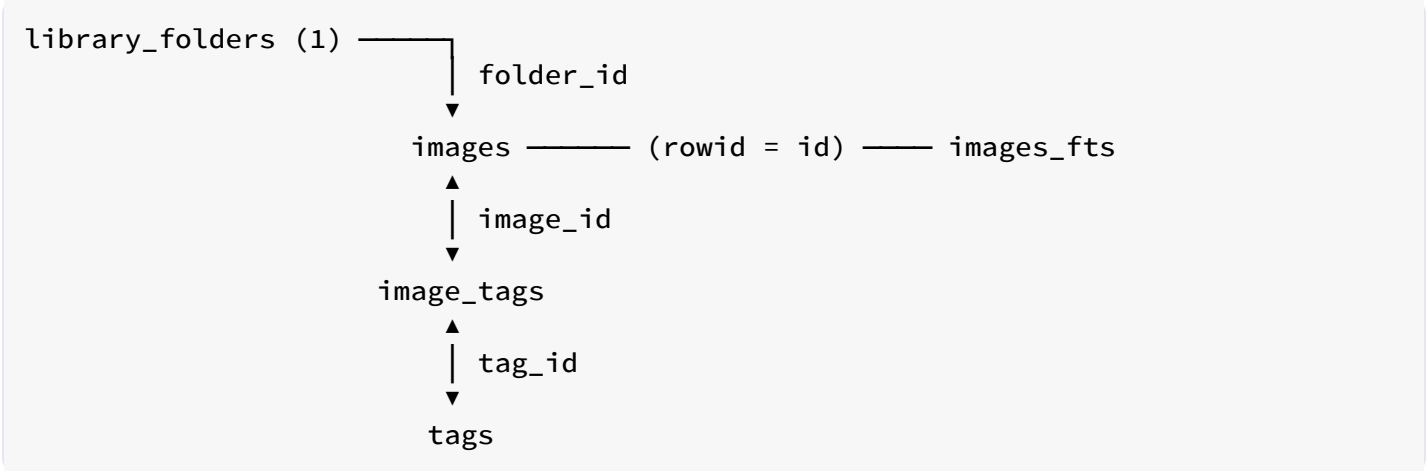
`schema_meta` (singleton, row `id = 1`)

Column	Type	Notes
id	INTEGER	Constant `1`.

Column	Type	Notes
magpie_version	TEXT	`CARGO_PKG_VERSION` when the DB was created.
schema_version	INTEGER	Bump on breaking schema changes.
created_at	INTEGER	Unix ms.

Used by `db::schema::apply` to decide whether the DB is fresh (no `schema_meta` table → run DDL) or upgradable (row present → check version).

Relationships



Global IDs

There is no packing scheme. `images.id` is a plain SQLite autoincrement primary key and is unique across the whole app. The IPC layer forwards it verbatim; the frontend treats it as opaque.

Migrations

`schema_meta.schema_version` is bumped when the schema changes. `db::schema::apply` runs upgrades one hop at a time; every hop lives in its own `migrate_vN_to_vN+1` helper.

Versions:

- **v1** — initial single-DB layout.
- **v2** — split `image_tags` by provenance. `source TEXT NOT NULL  CHECK (source IN ('auto','user'))` is added and the PK becomes `(image_id, tag_id, source)`. Existing rows have no provenance, so the migration marks them all as `'user'`; the next scan adds any `'auto'` rows the format handlers pick up alongside without disturbing user edits.
- **v3** — add automatic-AI-tagging bookkeeping to `images`: `ai_tagged_at INTEGER` and `ai_tag_hash TEXT`. Both nullable so existing rows migrate in place. The auto-tag pipeline (see Scanner and `core::auto_tag`) writes both columns once per successful classifier pass and compares `ai_tag_hash` against a per-image fingerprint on the next run to decide whether the image can be skipped.

Two legacy on-disk layouts are recognised at startup and imported one-shot into `magpie.db`:

1. **Pre-redesign** — `library.db` in the app-data dir.
2. **Per-folder redesign** — `registry.db` in the app-data dir plus `<folder>\.magpie\library.db` per registered folder.

See Database design for the migration algorithm. Legacy files are never deleted in place — they get a `.migrated-<yyyymmddThhmmss>` suffix so the user can inspect or restore them.

Tauri command reference

Every command below is registered in `src-tauri/src/lib.rs` inside `tauri::generate_handler![…]`. Signatures use `AppResult<T>` and camelCase JSON on the wire.

> **IDs.** Every `id: i64` an image command accepts or returns is the plain `images.id` primary key of the currently-open project's DB.
>
> **`NoProjectOpen`.** Every command below (except the `project::*` commands and `settings::*` commands) fails with `AppError::NoProjectOpen` when the user hasn't opened or created a project yet.

Project management

`current_project`

```
async fn current_project(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<Option<ProjectInfo>>;
```

Returns the currently-open project (`{ path, name }`) or `null` when none is open. The frontend calls this on mount to decide between the welcome screen and the main layout.

`create_project`

```
async fn create_project(
    services: State<'_, Arc<AppServices>>,
    path: String,
) -> AppResult<ProjectInfo>;
```

Create a fresh SQLite DB at `path` and swap it in as the current project. The `.magpie` extension is appended when `path` has none. Fails with `BadInput` if `path` already exists.

`open_project`

```
async fn open_project(
    services: State<'_, Arc<AppServices>>,
    path: String,
) -> AppResult<ProjectInfo>;
```

Open an existing project file and make it the current project. Fails with `PathNotFound` if the file isn't there.

`save_project`

```
async fn save_project(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<ProjectInfo>;
```

No-op (SQLite writes on every mutation). Kept so the menu item has a handler and the file's current descriptor can be surfaced back to the UI.

`save_project_as`

```
async fn save_project_as(
    services: State<'_, Arc<AppServices>>,
    path: String,
) -> AppResult<ProjectInfo>;
```

Copy the current DB to `path` using SQLite's online backup API, then reopen from the new location. Subsequent writes hit the new file.

`close_project`

```
async fn close_project(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<()>;
```

Drop the current project handle. The frontend then shows the welcome screen.

App settings

`get_app_settings`

```
async fn get_app_settings(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<AppSettings>;
```

Return the persisted `AppSettings` blob (theme, font size, language, last-project path, recent projects list).

`update_app_settings`

```
async fn update_app_settings(
    services: State<'_, Arc<AppServices>>,
    patch: AppSettingsPatch,
) -> AppResult<AppSettings>;
```

Merge a `{ theme?, fontSize?, language? }` patch and write the file atomically. Returns the new full settings.

Menu control

`set_menu_item_enabled`

```
fn set_menu_item_enabled(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    id: String,
    enabled: bool,
) -> AppResult<()>;
```

Toggle a menu item's enabled state. The frontend uses this to grey `edit_undo` / `edit_redo` when the corresponding stack is empty.

Library management

`add_library_folder`

```
async fn add_library_folder(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    path: String,
) -> AppResult<LibraryFolder>;
```

Canonicalise the path, insert a row into `library_folders`, and spawn a background scan. Emits `app://scan` progress events.

When **Settings → Auto-tag photos** is enabled (`AppSettings.aiAutoTag`) the scan is chained with an automatic-AI-tagging pass on the same folder — see `core::auto_tag::tag_folder`. That pass emits `app://auto-tag` progress events (see Events) and never runs on plain `rescan_folder` / `rescan_all`.

`remove_library_folder`

```
async fn remove_library_folder(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> AppResult<()>;
```

Delete the `library_folders` row. Because `images.folder_id` has `ON DELETE CASCADE`, all image rows, `image_tags`, and FTS rows for that folder are removed in the same transaction. **Source files on disk are untouched.**

`list_library_folders`

```
async fn list_library_folders(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<Vec<LibraryFolder>>;
```

Refreshes each folder's `isAvailable` flag by checking whether the root can be stat-ed, then returns the list. Availability is now purely about the filesystem; the DB itself is always present.

`rescan_folder`

```
async fn rescan_folder(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    id: i64,
) -> AppResult<ScanResult>;
```

Re-walks the specified folder. Incremental: only files with a newer `mtime` than what's in the DB get re-processed.

`rescan_all`

```
async fn rescan_all(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
) -> AppResult<Vec<ScanResult>>;
```

Rescan every *available* folder sequentially.

Images

`query_images`

```
async fn query_images(
    services: State<'_, Arc<AppServices>>,
    filter: Option<ImageFilter>,
    sort: Option<ImageSort>,
    page: Option<Pagination>,
) -> AppResult<Page<ImageSummary>>;
```

Plain `SELECT ... FROM images WHERE ... ORDER BY ... LIMIT ...` — one query against the single central DB. The result rows carry absolute paths, built by joining `images.rel_path` onto the folder root in Rust.

`get_image`

```
async fn get_image(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> AppResult<ImageDetails>;
```

Return full details for one image. `ImageDetails.userTags` and `ImageDetails.autoTags` come back separately so the DetailsPanel can render editable vs. read-only pills.

Side effect: if the source file's `mtime` is newer than what's stored in the row, the format handler is asked to re-read user metadata (title + tags from XMP/ `System.Keywords`) so that a Lightroom / Explorer edit made after import is picked up on next load. That re-read goes through `set_image_meta` , so any names the file now reports become **additive `'auto'` tags** — user tags are never overwritten. Fresh reads only update the row if the mtime moved forward; there is no periodic polling.

`update_image_metadata`

```
async fn update_image_metadata(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    id: i64,
    patch: MetadataPatch,
) -> AppResult<ImageDetails>;
```

Apply a metadata patch (title, tags, `tags_add` , `tags_remove`) to `magpie.db` . **All tag fields target the `'user'` source** — auto rows carried by the image are never inserted, removed, or renamed by this command. `tags` replaces the current user-tag list; `tags_add` inserts user rows for each name (no-op when already present as a user tag; an auto row with the same name is unaffected); `tags_remove` deletes user rows with matching names and leaves auto rows in place.

The source file is never touched. Rebuilds the row's FTS entry so subsequent search reflects the change. Emits `app://image-updated` .

`batch_update_metadata`

```
async fn batch_update_metadata(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    ids: Vec<i64>,
    patch: MetadataPatch,
) -> AppResult<Vec<i64>>;
```

Same as `update_image_metadata` but for many photos. Each `id` is patched inside its own transaction; the command returns the list of `ids` that succeeded (failures are logged and skipped).

`rename_image`

```
async fn rename_image(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    id: i64,
    new_filename: String,
) -> AppResult<ImageDetails>;
```

Rename a single file on disk and update its DB row. Steps:

1. Validate `new_filename` (non-empty, no path separators, no illegal Windows filename characters).
2. Compute the new absolute path in the same parent directory as the current one.
3. Refuse if a file already exists at the target path.
4. `std::fs::rename` on disk.
5. `queries::rename_image_row` — updates `images.filename`, `images.rel_path`, and `images.ext`, then rebuilds the FTS row. Fails with `BadInput` if the new `rel_path` would collide with another row in the same folder.
6. On DB failure, roll the FS rename back (best-effort).

Emits `app://image-updated` on success.

`delete_images`

```
async fn delete_images(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    ids: Vec<i64>,
    permanent: Option<bool>,
) -> AppResult<DeleteResult>;
```

Move to Recycle Bin (default) or permanently delete. Also removes the DB rows (via ON DELETE CASCADE) and cached thumbnails. Returns per-file success/failure.

Tags

`list_tags`

```
async fn list_tags(
    services: State<'_, Arc<AppServices>>,
    prefix: Option<String>,
) -> AppResult<Vec<TagStats>>;
```

Straight `SELECT` from the central `tags` table joined against `image_tags`. Aggregates both sources — an image with the same tag as both `'auto'` and `'user'` is counted once via `COUNT(DISTINCT it.image_id)`. Optional `prefix` narrows by `LIKE ? COLLATE NOCASE`.

`rename_tag`

```
async fn rename_tag(
    services: State<'_, Arc<AppServices>>,
    old_name: String,
    new_name: String,
) -> AppResult<()>;
```

Rename or merge. Operates on the shared `tags` vocabulary, so both `'auto'` and `'user'` rows follow the rename automatically. If `new_name` already exists, every `(image, source)` pair that pointed at `old_name` gets re-pointed at `new_name` (preserving provenance); the old vocabulary row is dropped. FTS rows for every affected image are rebuilt in the same transaction. **Source files are not touched.**

`delete_tag`

```
async fn delete_tag(
    services: State<'_, Arc<AppServices>>,
    name: String,
) -> AppResult<()>;
```

Remove a tag globally. `ON DELETE CASCADE` on the `image_tags` FK cleans out both `'auto'` and `'user'` rows in one go. **Source files are not touched.**

Smart collections

`list_smart_collections`

```
async fn list_smart_collections(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<Vec<SmartCollection>>;
```

`create_smart_collection`

```
async fn create_smart_collection(
    services: State<'_, Arc<AppServices>>,
    name: String,
    filter: ImageFilter,
) -> AppResult<SmartCollection>;
```

`delete_smart_collection`

```
async fn delete_smart_collection(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> AppResult<()>;
```

Smart collections live in the central `smart_collections` table next to everything else.

Thumbnails and image paths

`get_thumb_path`

```
async fn get_thumb_path(
    services: State<'_, Arc<AppServices>>,
    id: i64,
    size: ThumbSize,
) -> AppResult<String>;
```

Return the absolute path of a cached thumbnail; generate on demand if missing. Thumbnails are indexed by `images.id` and live under `%APPDATA%\com.magpie.app\thumbs\`.

`get_image_path`

```
async fn get_image_path(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> AppResult<String>;
```

Absolute path of the source image (folder root + `rel_path`), for the DetailsPanel inline preview via `convertFileSrc`.

Diagnostics

`log_frontend`

```
fn log_frontend(level: String, msg: String) -> AppResult<()>;
```

Push a log line into `app.log` from the renderer. `level` is one of `debug|info|warn|error`. Emitted with `target="frontend"` for easy filtering.

Magnifier window

`get_magnifier_context` / `set_magnifier_context` / `set_magnifier_current`

```
async fn get_magnifier_context(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<MagnifierContext>;

async fn set_magnifier_context(
    services: State<'_, Arc<AppServices>>,
    image_id: Option<i64>,
    filter: Option<ImageFilter>,
    sort: Option<ImageSort>,
) -> AppResult<()>;

async fn set_magnifier_current(
    services: State<'_, Arc<AppServices>>,
    image_id: Option<i64>,
) -> AppResult<()>;
```

The main window calls `set_magnifier_context` right before creating the Magnifier `WebviewWindow`; the popup calls `get_magnifier_context` on mount to know which image to display and which filtered set to walk. As the user navigates with the arrow keys, the popup calls `set_magnifier_current` so the "last shown" pointer survives close/re-open.

None of these three commands touch the DB, so they work whether or not a project is open.

Events (Rust → JS)

Event	Payload	Emitted by
`app://scan`	`ScanProgress`	`scanner::scan_folder` during scans.
`app://auto-tag`	`AutoTagProgress`	`core::auto_tag::tag_folder` during an auto tagging pass.
`app://ai-model-download`	`AiModelDownloadProgress`	`core::auto_tag::model_manager::ensure_d` while the CLIP model streams from HuggingF
`app://image-updated`	`i64` (image id)	Any command that mutates a row.

Event	Payload	Emitted by
`app://images-deleted`	`Vec<i64>`	`delete_images` after a successful batch.
`app://menu`	`String` (menu id)	Every native menu click; routed by `App.tsx`.

`AutoTagProgress` payload:

```
type AutoTagProgress = {
  folderId: number
  processed: number
  total: number
  currentPath: string | null
  tagsAdded: number   // cumulative tags attached this run
  skipped: number     // images skipped because fingerprint still matches
  finished: boolean
  // Populated only on `finished: true` when the pass could not run
  // (usually because the CLIP model has not been downloaded yet).
  error?: string | null
}
```

On `finished: true` the frontend invalidates the `images`, `tags`, and `image` query keys so the sidebar tag cloud and any open details panel pick up the newly-attached tags. When `error` is non-null the status bar surfaces it as a small warning (“*AI model not downloaded*”) but otherwise treats the pass as a no-op.

`AiModelDownloadProgress` payload:

```
type AiModelDownloadProgress = {
  currentFile: string       // e.g. "model.safetensors"
  currentBytes: number      // bytes fetched for currentFile so far
  currentTotal: number      // Content-Length for currentFile
  totalBytes: number        // cumulative bytes across all files
  totalExpected: number     // AI_MODEL_TOTAL_BYTES (~607 MB)
  finished: boolean
  error?: string | null
}
```

Ticks are throttled to ~200 ms so the download dialog can draw a smooth progress bar. On `finished: true` the frontend re-runs `check_ai_model_status` — as soon as `status.ready` flips true the **Auto-tag photos** dialog enables its toggle.

AI-model management commands

Backed by `core::auto_tag::model_manager` — see Backend → `core/auto_tag/`.

- `check_ai_model_status() -> AiModelStatus` — cheap `stat`-only probe of `<app_data_dir>/models/clip/`. Reports whether the safetensors weights, tokenizer JSON, and pre-computed vocab embedding cache are on disk, plus the total expected download size.
- `download_ai_model()` — idempotently streams every missing file from HuggingFace under an `AppServices.auto_tag_gate` lock (so a folder-add auto-tag pass and a manual download can't compete for the same partial file). Verifies each file's SHA-256 before promoting the `.part` file to its final name. Emits `app://ai-model-download` while it runs.
- `clear_ai_model()` — removes every file under `models/clip/`, including the embedding cache. Used by the Settings dialog's **Remove model files** button.

Menu commands

- `set_menu_item_enabled(id, enabled)` — used for context-sensitive items (`edit_undo`, `edit_redo`, `view_magnifier`).
- `set_menu_item_label(id, label)` — legacy hook that used to suffix a `✓` on the **Settings → Auto-tag photos** entry. The entry is now a static "…" menu item that opens the Auto-tag dialog; the check-state lives entirely inside the dialog, so `App.tsx` no longer calls this command for that item.

Scanner algorithm

Objective

Given a library folder path, populate `magpie.db` with a row per supported file, extract metadata, generate thumbnails, and keep the DB in sync with the filesystem on subsequent runs — all while staying responsive.

Every path stored in the DB is **folder-relative** (forward slashes). The scanner joins root + relative path when it needs the actual filesystem location.

Pipeline

```
walk()  →  filter()  →  diff()  →  extract()  →  upsert()  →  thumbnail()
   │           │           │           │           │              │
jwalk::   ext-in-       mtime         rayon        DB           thumb
WalkDir   IMAGE_EXTS    match?       parallel   serialised    pipeline
 par      ↳ skip        ↳ skip                   (mutex)
 walk    others         else read
```

Progress events (`app://scan { folder_id, done, total, current }`) are emitted every N files (default 20).

Stage 1 — walk

`jwalk::WalkDir::new(root).parallelism(Parallelism::RayonNewPool(n))` walks the folder tree in parallel across all cores, streaming `DirEntry` values. Symlinks are followed (with cycle detection); hidden files are respected per OS convention.

Stage 2 — filter

An entry is kept iff:

- It's a regular file (not a directory, socket, or device).

- Its lowercase extension is in `core::IMAGE_EXTS`: `jpg jpeg jpe jfif jif png gif bmp tif tiff webp heic heif cr2 cr3 nef arw dng raf orf pef x3f`.
- Its filename doesn't start with `.` (dotfiles).

Discarded entries increment a counter used for the progress denominator but otherwise do no work.

Stage 3 — diff

For each kept entry, Magpie looks up the *folder-relative* path in the `images` table:

```
SELECT id, mtime_ms FROM images
 WHERE folder_id = ?1 AND rel_path = ?2
```

Leftover `.magpie/library.db` files from a failed migration off the previous per-folder layout are excluded from the walk so the scanner doesn't try to import them.

- **New file** (no row): full extract required.
- **Existing, mtime unchanged**: skip.
- **Existing, mtime changed**: partial re-extract (metadata + hash), leaving other columns intact.

The lookup is O(log n) thanks to the UNIQUE index on `(folder_id, rel_path)`.

Stage 4 — extract

For each file that needs work, the following runs on a rayon worker:

1. **Stat** — `fs::metadata(path)` for `size_bytes` and `mtime_ms`.
2. **Content hash** — stream the file through `XXH3_128`. On an SSD this is disk-bound at ~2 GB/s; on HDD it's the bottleneck.
3. **Handler lookup** — `registry.for_ext(ext)` picks the `FormatHandler` for the file's extension.
4. **Technical read** — the handler's `read_technical` produces ordered key/value pairs; those we recognise fold into `taken_at`, `camera_make`, `camera_model`, `width`, `height`.
5. **User meta** — `metadata::read::read_all` combines the handler's `read_user` with any legacy sidecar for `title` and `tags`.

Any per-file failure is logged (`WARN`) and doesn't abort the folder scan. The file gets a row with whatever metadata succeeded; missing fields are left NULL.

Stage 5 — upsert

The central `magpie.db` is the single writer. Scanner tasks take short borrows of the `Db` mutex — one upsert per statement — so scan work interleaves with UI queries without long-lived locks.

The upsert uses `(folder_id, rel_path)` as the conflict key:

```
INSERT INTO images (folder_id, rel_path, filename, ext,
                    size_bytes, mtime_ms, imported_at, missing)
VALUES (?, ?, ?, ?, ?, ?, ?, 0)
ON CONFLICT(folder_id, rel_path) DO UPDATE SET
    filename    = excluded.filename,
    ext         = excluded.ext,
    size_bytes  = excluded.size_bytes,
    mtime_ms    = excluded.mtime_ms,
    missing     = 0;
```

After the row lands, tags are reconciled by `set_image_meta`. Auto tags are **additive-only**: each name the file itself reports is inserted with `source = 'auto'` **only when the image doesn't already carry that name in either source**. Nothing is deleted, so auto tags that vanished from the file's metadata since the last scan stay in the DB, and user tags typed inside Magpie always survive. See Schema › image_tags for the storage model. All of this happens in the same transaction as the image upsert.

The FTS5 row is rebuilt via `rebuild_fts_row_tx` at the very end.

Stage 6 — thumbnails

Once a row is in the DB, the scanner enqueues a thumbnail task for that image id. Thumbnails run on the same rayon pool but with lower priority so they don't starve extraction.

`ensure_thumbnails(cache_dir, src_path, image_id)`:

1. For each size (small, medium):
   - Path = `cache_dir / "<image_id>-<size>.webp"`. `image_id` is the plain `images.id` primary key — globally unique because Magpie has a single central DB.
   - Skip if the thumbnail's mtime ≥ the source's mtime.

2. Decode the source with `image::open`.
3. Resize with `fast_image_resize::Resizer` (Lanczos3, SIMD).
4. Encode with `webp::Encoder::new(&image).encode(80)`.
5. Write bytes atomically (temp + rename).

Failures at any stage are logged and don't affect the DB row.

Handling deletions

If a file that was in the DB is no longer found on disk during a scan, the scanner does **not** delete the row automatically — the `missing` flag in the `images` table exists to mark it (v1: not exposed in UI). A future contribution can add an explicit "Clean up missing photos" UI action.

Stage 7 — Auto-tag pass (opt-in)

When **Settings** → **Auto-tag photos** is on (`AppSettings.aiAutoTag`), `commands::library::add_library_folder` chains an automatic AI classification pass onto the tail of a successful scan:

```
scanner::scan_folder(...)  →  auto_tag::tag_folder(...)
```

Structurally identical to the scanner:

- One `spawn_blocking` task per image, bounded by a `Semaphore::new(cpus)`.
- Progress emitted on the `app://auto-tag` event as an `AutoTagProgress` payload every 5 images (and once on start / finish).
- A single `tokio::sync::Mutex<()>` on `AppServices.auto_tag_gate` serialises AI passes across folders — if the user drops several folders in quick succession they queue up FIFO instead of competing for CPU. Filesystem scans stay parallel.

Per-image loop (`core::auto_tag::tag_one`):

1. Enumerate candidates via `queries::list_auto_tag_candidates(folder_id)`. Each candidate carries a "fingerprint" — the row's `content_hash` if the handler wrote one, else `mtime_ms.to_string()`.
2. If `ai_tag_hash == fingerprint`, skip — the image hasn't changed since the last successful pass. Counted as `skipped` in the progress payload.

3. Ensure the small thumbnail exists (reuse the scanner's `thumbnail::ensure_thumbnails`); if the format isn't decodable by the `image` crate, mark the row as tagged with zero suggestions so we don't try again next run.
4. Read the thumbnail bytes and call `ImageClassifier::classify(&bytes)`. In production the trait is implemented by `core::auto_tag::clip_classifier::ClipClassifier`, which runs OpenAI **CLIP-ViT-B/32** on the CPU via `candle` (https://github.com/huggingface/candle) and cosine-ranks the resulting image embedding against a pre-computed matrix of text embeddings for the ~1 000-word bundled photo vocabulary (see `core/auto_tag/`). Tests use the deterministic `MockClassifier` in `core::auto_tag::classifier.rs` — swap in an `Arc<dyn ImageClassifier>` at the `tag_folder_with` entry point. `tag_folder` refuses to spawn the classifier when the CLIP model files aren't on disk — it emits one "finished with error" progress event instead of running the pass. The `AppServices.auto_tag_gate` also protects the model download against a concurrent auto-tag pass.
5. Filter suggestions by `classifier.min_confidence()`, sort by descending confidence, cap at `classifier.max_tags_per_image()`.
6. Attach the surviving names via `queries::add_auto_tags_for_image(image_id, names)`, which writes them as `'auto'`-source tags in a short transaction and rebuilds the FTS row. Names the image already carries (in either source) are a no-op, so the row count stays sane on rerun and any tag the user has typed themselves is left alone. The tags therefore render under the read-only **Automatic tags** section in the details panel, next to XMP-derived tags.
7. `queries::mark_image_ai_tagged(image_id, fingerprint, now_ms)` stamps `ai_tagged_at` and `ai_tag_hash` on the row.

Auto-tag never runs during plain `rescan_folder` or `rescan_all` today — only on the very first scan of a newly-added folder. A follow-up can add an explicit "run AI on this folder now" command if we need it.

Incremental performance

The diff step is what makes rescans fast:

- 50 000 photos, no changes: `~2 seconds` (mostly the DB lookup and a stat call per file).
- 50 000 photos, 100 new: `~5 seconds` (mostly hashing + XMP read for the 100).
- 50 000 photos, cold cache (thumbnails missing): `~2 minutes` (bounded by encoding).

Thumbnail pipeline

Sizes

Two cached sizes per image, both stored as WebP quality 80:

Enum name	Long-edge px	Used by
`Small`	~256	`ImageGrid` tiles (main use case).
`Medium`	~512	(Reserved for a future zoom-in mode.)
`Large`	~1024	(Currently only generated on request.)

Aspect ratio is preserved — the short edge is proportional. A 6000×4000 source becomes a 256×170 small and 512×341 medium.

Location

Files live in `%APPDATA%\com.magpie.app\thumbs\`:

```
<id>-small.webp
<id>-medium.webp
```

Using the image id (rather than a hash of the source path) keeps the cache invariant across file renames: as long as Magpie still knows about the image, the thumbnail is reusable.

Generation

```
pub fn ensure_thumbnails(cache_dir: &Path, src: &Path, image_id: i64) ->
Result<()> {
    let src_mtime = fs::metadata(src)?.modified()?;
    for size in &[ThumbSize::Small, ThumbSize::Medium] {
        let out = thumb_path(cache_dir, image_id, *size);
        if is_fresh(&out, src_mtime) { continue; }
        let img = image::open(src)?;
        let target = pick_target_size(&img, *size);
        let thumb = resize_rgba(&img.to_rgba8(), target);
        save_webp(&thumb, &out)?;
    }
    Ok(())
}
```

Details:

- `image::open` handles JPEG, PNG, GIF, TIFF, WebP, HEIC (with the `image` crate's HEIC feature enabled). For unsupported RAW formats, `open` returns an error and the thumbnail is skipped (grid renders a placeholder).
- `resize_rgba` uses `fast_image_resize::Resizer` with the `Lanczos3` filter. On x86_64 this dispatches to SSE4/AVX2 SIMD automatically.
- `save_webp` writes to `<out>.tmp` then renames — atomic, so a crash mid-encode never leaves a corrupt WebP.

Freshness

A thumbnail is considered fresh iff its file mtime is ≥ the source file's mtime. Magpie never modifies the source, so editing tags or titles doesn't invalidate a thumbnail. If a user replaces the file externally (bumping its mtime), the next scan regenerates.

Deletion

`delete_thumbnails(cache_dir, image_id)` is called from `delete_images`. It removes every size:

```
for size in [Small, Medium, Large] {
    let _ = fs::remove_file(thumb_path(cache_dir, image_id, size));
}
```

Errors are ignored — a missing thumb is the desired state.

Cache sizing

The `Small` WebP averages 4–8 KB per photo; `Medium` averages 16–32 KB. Total cache size on a 50 000-photo library is roughly 1–2 GB.

There's no automatic eviction in v1. Users who want to trim the cache can delete `%APPDATA%\com.magpie.app\thumbs\` — Magpie regenerates on demand.

Async model

`ensure_thumbnails` is CPU-bound (decode + resize + encode). It's called from Tauri commands via `tauri::async_runtime::spawn_blocking` so it doesn't stall a Tokio worker. The scanner enqueues thumb tasks on the rayon pool to overlap I/O with compute.

Metadata read path

Entry point

```
pub fn read_all(registry: &FormatRegistry, path: &Path) ->
AppResult<ImageMetaFromFile>;
```

Called from:

- The scanner (initial scan and rescans), to populate the `images` row in `magpie.db`.
- `get_image` (Tauri command), when the source file's `mtime` has moved forward since the row's cached `mtime_ms`.

`ImageMetaFromFile` is the union of everything we can pull from disk that the DB stores:

```
pub struct ImageMetaFromFile {
    pub title:        Option<String>,
    pub tags:         Vec<String>,      // deduped, in insertion order
    pub taken_at:     Option<i64>,      // ms since Unix epoch
    pub camera_make:  Option<String>,
    pub camera_model: Option<String>,
    pub width:        Option<u32>,
    pub height:       Option<u32>,
}
```

`ImageMetaFromFile` is used **only on import** (first scan of a file, or on mtime bump). After the row exists in the DB, subsequent edits stay in the DB via `queries::apply_metadata_patch`.

Stages

1. Ask the handler

```
let handler = registry.for_ext(ext).ok_or(AppError::UnsupportedFormat)?;
let user      = handler.read_user(path)?;
let technical = handler.read_technical(path);
```

`read_user` returns the editable surface (title + tags). `read_technical` returns the ordered `TechnicalMeta` list used by the UI and by the scanner for dimensions / EXIF-derived fields (camera, taken_at). See File formats.

Each handler decides how to fetch this. Native handlers (JPEG, PNG, WebP, GIF, TIFF) use the shared `xmp_packet::parse_xmp` helper; stub handlers (HEIF, video, PDF, RAW, basic raster) implement `read_technical` from magic bytes and defer `read_user` to the Windows Shell fallback.

2. Windows Shell property store (import-only, Windows-only)

On Windows, if `handler.read_user` didn't return a title or tags (typical for RAW, HEIC, MP4, PDF), `win_shell::read_user_meta` is called. It uses `SHGetPropertyStoreFromParsingName` to open the file's shell property store read-only and asks for `System.Title`

- `System.Keywords`. This is how tags that a user typed into Explorer's *Properties → Details* dialog get imported into Magpie on first scan.

`win_shell` is strictly read-only after the redesign — there is no matching write path.

3. Legacy sidecar XMP

```
let sidecar = read_sidecar(&sidecar_path_for(path)).ok();
```

Sidecar path is `<file basename>.xmp`. Sidecars are a **legacy compatibility** path: Magpie no longer produces them, but older Magpie installs and other tools (Lightroom, digiKam) did. If a sidecar file exists it's parsed with the same XMP parser used for embedded packets and its fields are unioned into the result.

4. Merge and fold into DB row

`apply_user_meta` folds the handler's `UserMeta`, then the Windows Shell result, then the legacy sidecar (in that order — sidecar wins if it exists, matching Lightroom conventions on read). The merged `ImageMetaFromFile` is then written to `magpie.db` via `queries::set_image_meta`.

XMP parser

`xmp_packet::parse_xmp(bytes) -> XmpUserMeta` is a hand-written streaming parser built on `quick_xml`. It normalises casing/namespaces because tools in the wild are notoriously inconsistent (`<X:XMPMETA>` , `<x:xmpmeta>` , and mixed-case in the same file).

Recognises both `dc:subject` (standard) and `MicrosoftPhoto:LastKeywordXMP` (Windows Explorer); unions the two sets, deduping case-insensitively while preserving the first observed casing. Handles both `<rdf:Alt>` and `<rdf:Seq>` inside `<dc:title>` ; the `x-default` language item wins if present, otherwise the first item.

FS-refresh check on `get_image`

Inside the `get_image` command, before returning:

```
if let Ok(fs_meta) = std::fs::metadata(&abs) {
    let disk_mtime = fs_meta.modified()?.duration_since(UNIX_EPOCH)?.as_millis()
as i64;
    if disk_mtime > row.mtime_ms {
        let fresh = read_all(&registry, &abs)?;
        queries::set_image_meta(&mut conn, id, &fresh)?;
    }
}
```

Simple mtime comparison. If the file was modified after import, re-read metadata and overwrite the row. This is how an external tag edit (Windows Explorer, Lightroom export) becomes visible in Magpie on next click, without needing a full library rescan.

Note: the mtime bump wipes Magpie-side edits with whatever's currently in the file. In the new model, once the file is in the DB, edits should happen in Magpie. If you edit tags in Explorer after Magpie has been in charge, those Explorer edits win the next time `get_image` is called for that file — because the file's mtime is now newer, so Magpie re-reads it. Users who want to switch tagging tools mid-flight should be aware of this trade-off.

Database design

Magpie stores every piece of user-visible metadata in a single SQLite file at `%APPDATA%\com.magpie.app\magpie.db`. Source files on disk are never modified.

This chapter captures the rationale, the schema shape, and the migration story from the two earlier on-disk layouts that Magpie has carried users through.

Why a single central DB

We evaluated three options:

1. **Central DB in `%APPDATA%`** — the current design.
2. **Per-folder DB inside `<folder>/.magpie/library.db`** — the previous design.
3. **Metadata written back into source files** — the pre-redesign design.

Option 3 was abandoned because too many formats (RAW, PDF, MP4, HEIC, …) don't accept sidecar-free tag writes on Windows without elevation and format-specific tricks, and the roundtrip cost was crushing scan performance.

Option 2 solved the write problem but introduced its own set of issues:

- Hidden `.magpie` folders inside the user's photo trees.
- Cloud-sync clients (OneDrive, Dropbox, iCloud) racing against Magpie for the DB file.
- Cross-folder search required attaching every registered library to a single connection at startup, with a hard cap at 10 attached DBs in the bundled SQLite.
- Packed *global* IDs (`folder_id * 1_000_000_000 + local_id`) to paper over the fact that each folder's `images.id` starts at 1.

Option 1 keeps the "read-only source files" invariant from option 2 but drops every one of its downsides:

- One file. Trivial to back up, trivial to reason about.
- Lives outside the user's photo trees.
- Cloud-sync clients don't touch `%APPDATA%` by default, so the race disappears.
- Cross-folder search is a plain `SELECT ... FROM images` with no attach dance.
- `images.id` is a normal autoincrement PK and is globally unique for free.

Trade-off: the DB doesn't travel with a folder. If the user moves a photo folder to another PC, they need to add it fresh on the target machine and re-tag (or re-scan to pick up whatever tags

the files already embed). This matches how every other photo manager on the market works.

Schema

The full DDL lives in `src-tauri/src/db/schema.sql`. See the Database schema chapter for column-by-column tables. Highlights:

- `library_folders` — registered folder roots, absolute canonical paths.
- `images` — one row per scanned file, `(folder_id, rel_path)` UNIQUE, `folder_id` FK to `library_folders` with ON DELETE CASCADE.
- `tags` — global vocabulary; `name` UNIQUE COLLATE NOCASE so case variants merge automatically.
- `image_tags` — many-to-many join, both FKs cascade on delete.
- `images_fts` — FTS5 over `title`, `filename`, `tags`.
- `smart_collections`, `app_settings`, `schema_meta` — small bookkeeping tables.

Paths

`images.rel_path` is folder-relative with forward slashes. Absolute paths are built at query time by joining with `library_folders.path` in Rust. Moving a folder root only requires updating one row in `library_folders`.

IDs

`images.id` is a plain SQLite autoincrement `INTEGER PRIMARY KEY`. It's globally unique because there's only one central DB. The IPC layer forwards it to the frontend as `ImageSummary.id` verbatim; JavaScript's `Number.MAX_SAFE_INTEGER` gives us $2^{53} - 1$ values, which is nine quadrillion images — more than enough.

Connection layer

`crate::db::Db` wraps a single `rusqlite::Connection` behind `Arc<Mutex<Connection>>` for `Send + Sync`. Every command goes through one of two shortcuts:

- `db.with_conn(|conn| …)` — read/write borrow for a single statement or read-only query.

- `db.with_conn_mut(|conn| …)` — the closure receives `&mut Connection` so it can start a transaction.

The mutex is only held for the closure's lifetime; long-running scans and thumbnail generation run outside it and touch the DB in short bursts.

Read-only source files

Format handlers implement two methods: `read_technical` (returns the label/value pairs shown in the "File info" section of the details panel) and `read_user` (returns title + tags). There is no `write_user`. On first scan the scanner reads whatever the file already carries — native XMP for JPEG/PNG/WebP/GIF, the Windows Shell property store for everything else, plus legacy Lightroom `.xmp` sidecars — and imports it into `magpie.db`. From that point on the DB is the sole source of truth; the file itself is left untouched.

Migration from earlier layouts

`db::migrate::open_or_migrate(app_data_dir)` runs on every launch:

1. **`magpie.db` exists** — open it, apply any pending schema migrations, done.
2. **`registry.db` exists (per-folder design)** — create a fresh `magpie.db`, then for each row in the old registry:
   - Read `<folder>\.magpie\library.db` read-only.
   - Copy the folder row into `magpie.db.library_folders` (new PK).
   - Copy every image row, remapping `folder_id`.
   - Copy tags via name (insert-or-ignore into the central `tags` table) and rebuild `image_tags`.
   - Rebuild the FTS row for each imported image.
   - After all folders succeed, rename `registry.db` to `registry.db.migrated-<yyyymmddThhmmss>` and delete each migrated `<folder>\.magpie` directory.
3. **`library.db` exists (pre-redesign)** — same idea, but the source is a single old-central DB. Absolute paths in the legacy `images.path` column are converted to folder-relative using `library_folders.path`. Legacy file is renamed to `library.db.migrated-<ts>`.
4. **Nothing exists** — create empty `magpie.db`.

Migration is idempotent: `magpie.db` is populated first, source files are only renamed after everything commits. A mid-migration crash leaves the legacy files untouched and the next launch retries from step 1.

What the redesign removed

Rust:

- `db/pool.rs` — the `LibraryPool` and its ATTACH management.
- `db/library.rs` / `db/registry.rs` — schema modules for the per-folder and registry DBs; consolidated into `db/queries.rs`.
- `db/search.rs` — the `UNION ALL`-across-attached-schemas query builder; replaced by a plain `SELECT ... FROM images`.
- `db/legacy_migration.rs` — replaced by `db/migrate.rs` which handles both legacy layouts.
- `db::pack_global_id` / `db::unpack_global_id`.
- `commands::library::check_folder_sync_risk` and `SyncRiskWarning` — with the DB in `%APPDATA%`, cloud sync on the photo folder is safe.

TypeScript:

- `SyncRiskWarning` type + `checkFolderSyncRisk` IPC wrapper.
- The `confirm` dialog in `TopBar.tsx` that gated adding folders on cloud drives.

Diagnostics

Two diagnostic examples are shipped:

- `cargo run -q --example dump_meta` — dumps the first few rows from `magpie.db` and re-runs the metadata read pipeline on each.
- `cargo run -q --example dump_tag_usage` — prints the images associated with a tag (`$env:MAGPIE_QUERY_TAG = 'sunset'`).

Both open `magpie.db` read-only and are safe to run while Magpie is running.

File formats

Magpie's metadata read path is powered by a small trait-based plug-in system rather than a hard-coded `match ext { … }`. Every handler is **read-only** — see Database redesign and Adding a format handler.

The `FormatHandler` trait

Every supported file type is represented by a Rust type that implements the trait declared in `src-tauri/src/core/formats/mod.rs`:

```
pub trait FormatHandler: Send + Sync {
    fn name(&self) -> &'static str;
    fn extensions(&self) -> &'static [&'static str];
    fn kind(&self) -> FormatKind;

    fn read_technical(&self, path: &Path) -> TechnicalMeta;
    fn read_user(&self, path: &Path) -> AppResult<UserMeta>;
}
```

- `name` — human-readable label shown in the details panel (e.g. `"JPEG"`).
- `extensions` — lowercase extensions this handler owns (`&["jpg", "jpeg"]`).
- `kind` — image / video / document / other. Used to classify entries and skip inappropriate operations (thumbnails, previews).
- `read_technical` — the read-only metadata shown at the bottom of the details panel. An ordered list of `(label, value)` pairs.
- `read_user` — extract title + tags from the file (XMP, EXIF, container-specific atoms). Called on import.

The `FormatRegistry`

`FormatRegistry` maps each lowercase extension to exactly one handler. It's constructed once at startup, wrapped in `Arc`, and carried by `AppServices`.

Handlers register themselves in `FormatRegistry::new()`. The registry is the sole authority for:

- **Which extensions are scannable.** `scanner` calls `registry.for_ext(ext).is_some()` to decide whether to index a file.

- **Where technical/user metadata comes from.** `metadata::read::read_all` builds an `ImageMetaFromFile` by calling `handler.read_technical(path)` and `handler.read_user(path)`, then merging in the Windows Shell property store result and any legacy `.xmp` sidecar.

First-scan tag import

On the *first* time the scanner sees a file, `read_user` and the Windows Shell fallback are consulted so tags that a previous tagger (Lightroom, Adobe Bridge, Explorer's *Properties → Details*) already put in the file are imported into the folder's `library.db`. After that, the DB is the source of truth and the file is never read again for user metadata unless its `mtime` moves forward.

Handler catalogue

The "Reads tags" column describes what the native handler can pull out during a first-scan import. Formats whose native handler can't parse tags still get scanned into the DB — the Windows Shell fallback usually fills in title/keywords on Windows.

Handler	Kind	Extensions	Reads tags?	Notes
JPEG	Image	`.jpg`, `.jpeg`	✅ XMP APP1	
PNG	Image	`.png`	✅ iTXt XMP	
WebP	Image	`.webp`	✅ RIFF XMP	
GIF	Image	`.gif`	✅ (GIF89a)	Adobe XMP-in-GIF Application Extension
TIFF	Image	`.tif`, `.tiff`	✅ tag 700	

Handler	Kind	Extensions	Reads tags?	Notes
HEIC / HEIF / AVIF	Image	`.heic`, `.heif`, `.hif`, `.avif`	❌ (Shell import)	Native read of box-level EXIF for `taken_at` only
JPEG XL	Image	`.jxl`	❌ (Shell import)	
JPEG XR / HD Photo	Image	`.jxr`, `.wdp`	❌ (Shell import)	
JPEG 2000	Image	`.jp2`, `.j2k`, `.j2c`, `.jpx`, `.jif`	❌ (Shell import)	
PSD	Image	`.psd`, `.psb`	❌ (Shell import)	
PDF	Document	`.pdf`	❌ (Shell import)	
MP4/MOV family	Video	`.mp4`, `.m4v`, `.mov`, `.3gp`, `.3gpp`, `.3g2`	❌ (Shell import)	
Matroska / WebM	Video	`.mkv`, `.mks`, `.mka`, `.webm`	❌ (Shell import)	
AVI	Video	`.avi`	❌ (Shell import)	
WMV/ASF	Video	`.wmv`, `.asf`	❌ (Shell import)	
MPEG-TS	Video	`.ts`, `.mts`, `.m2ts`	❌ (Shell import)	
Canon RAW	Image	`.cr2`, `.cr3`, `.crw`	❌ (Shell import)	
Nikon RAW	Image	`.nef`, `.nrw`	❌ (Shell import)	
Sony RAW	Image	`.arw`, `.sr2`, `.srf`, `.arq`	❌ (Shell import)	

Handler	Kind	Extensions	Reads tags?	Notes
Fuji RAW	Image	`.raf`	❌ (Shell import)	
Olympus RAW	Image	`.orf`, `.ori`	❌ (Shell import)	
Panasonic RAW	Image	`.rw2`, `.rwl`	❌ (Shell import)	
Pentax RAW	Image	`.pef`	❌ (Shell import)	
Samsung RAW	Image	`.srw`	❌ (Shell import)	
Sigma RAW	Image	`.x3f`	❌ (Shell import)	
Adobe/generic DNG	Image	`.dng`	✅ (TIFF-family)	
BMP	Image	`.bmp`, `.dib`	❌ (Shell import)	
OpenEXR	Image	`.exr`	❌	Library-only tags after import
Radiance HDR	Image	`.hdr`, `.hdp`	❌	Library-only tags after import
SVG	Image	`.svg`	❌	Library-only tags after import

"Shell import" means: the format's native `read_user` returns empty, and `metadata::read::read_all` then asks Windows' `SHGetPropertyStoreFromParsingName` for `System.Title` + `System.Keywords`. So on Windows the tag import still works for any format Explorer's *Details* pane recognises. On other platforms (future) these formats will simply start empty.

Where the code lives

```
src-tauri/src/core/formats/
├── mod.rs           # FormatHandler trait + FormatRegistry + TechnicalMeta / UserMeta
├── common.rs        # Shared: EXIF → TechnicalMeta, dimensions, verbatim-prefix helpers
├── xmp_packet.rs    # XMP parser (read-only)
├── win_shell.rs     # IPropertyStore fallback reader (Windows only; stub elsewhere)
├── jpeg.rs          # native XMP APP1 reader
├── png.rs           # native iTXt XMP reader
├── webp.rs          # native RIFF XMP reader
├── gif.rs           # native Application-Extension XMP reader (GIF89a)
├── tiff.rs          # native tag 700 XMP reader
└── stubs.rs         # everything else, one struct per family
```

`stubs.rs` deliberately groups similar formats so adding a new “read technical + Shell-import user meta” format is one struct + one line in `FormatRegistry::new`.

Adding a format handler

Format handlers are the plug-in point for supporting a new file type. **All handlers are read-only** after the DB redesign — Magpie never writes back into source files. See Database redesign.

This page walks through the four-step recipe using a fictitious "MyFormat" (`.myf`) as an example.

1. Pick the right home

- Add a new struct to `src-tauri/src/core/formats/stubs.rs` if the handler only pulls technical metadata via `imagesize` / magic bytes, or
- Create a dedicated module `src-tauri/src/core/formats/myformat.rs` if you need a real parser (like `jpeg.rs` , `png.rs` , `tiff.rs`).

2. Implement `FormatHandler`

```rust
use super::common::{self, TechnicalMeta};
use super::{FormatHandler, FormatKind, UserMeta};
use crate::error::AppResult;
use std::path::Path;

pub struct MyFormat;

impl FormatHandler for MyFormat {
    fn name(&self) -> &'static str {
        "MyFormat"
    }
    fn extensions(&self) -> &'static [&'static str] {
        &["myf"]
    }
    fn kind(&self) -> FormatKind {
        FormatKind::Image
    }

    fn read_technical(&self, path: &Path) -> TechnicalMeta {
        let mut t = TechnicalMeta::default();
        common::append_file_basics(&mut t, path);
        // ... push more pairs specific to this format ...
        t
    }

    fn read_user(&self, path: &Path) -> AppResult<UserMeta> {
        // Native handlers can parse an embedded XMP packet here.
        // Stubs that rely on the Windows Shell fallback just
        // return an empty result and let read_all pick up the
        // slack.
        let _ = path;
        Ok(UserMeta::default())
    }
}
```

That's the whole trait. There's no `write_user`, no `can_write_tags`. If a file's format ever needs bespoke read-only handling of embedded tags (say, a proprietary MP4 atom), do it inside `read_user` — that's the only user-metadata seam we still expose.

3. Register it

Open `src-tauri/src/core/formats/mod.rs` and add the handler to `FormatRegistry::new`:

```rust
registry.register(Arc::new(myformat::MyFormat));
```

For a stub, add it to `stubs::register` (already called from `FormatRegistry::new`).

4. Write tests

Add a scanner-facing test that confirms:

- Files with the new extension are picked up by the walk.
- `handler.read_technical(path)` returns sensible values for a minimal fixture.
- `handler.read_user(path)` returns the tags Magpie should import on first sight (if the format supports embedded metadata).

The `registry_recognises_every_expected_extension` test in `src-tauri/tests/format_registry.rs` is the right place to prove the extension is known.

5. Update the docs

- Add a row to the handler catalogue on File formats.
- Add the extension to the user-manual Supported file formats page.

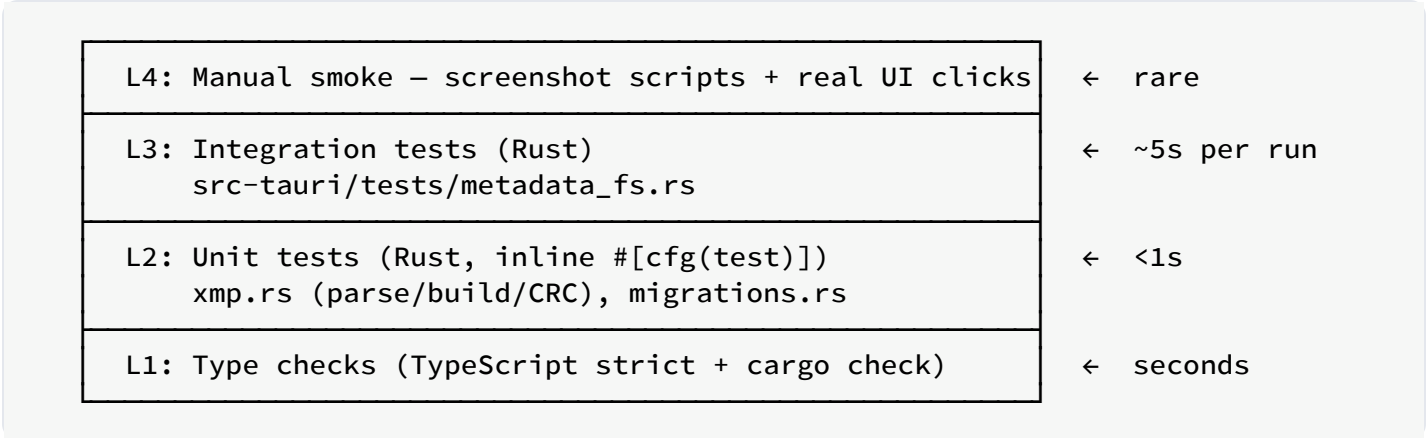
Design invariants

While writing a new handler, keep these invariants in mind:

- **Never write to the source file.** The DB is the source of truth for tags and titles. Any hypothetical need to write back should be discussed as an architectural change, not sneaked into a handler.
- **No blocking I/O on hot paths.** Handlers are invoked from the scanner (parallel) and the `get_image` command (foreground); restrict work in `read_technical` to what's cheap enough to run during a scan.
- **Failures are non-fatal.** If reading a specific field fails, return an empty `UserMeta` / partial `TechnicalMeta` — the caller will still insert the row with whatever succeeded.

Testing strategy

Layers



L1 — type checks

- `cargo check --workspace` for the Rust side (no unused warnings allowed in CI).
- `tsc -b --pretty false` for the frontend; strict mode enabled in `tsconfig.json` (`noUncheckedIndexedAccess`, `noImplicitAny`, `strictNullChecks`).

Both are cheap and run before every commit.

L2 — Rust unit tests

Colocated with modules in `#[cfg(test)] mod tests { … }`. Examples:

- `core/metadata/xmp.rs` — parser tests for Windows Explorer tags, Microsoft-only keyword lists, case variants, attribute-form values. Also covers the JPEG APP1 walker and (in integration tests) the PNG iTXt CRC.
- `db/queries.rs` — smoke tests for FTS row rebuild.

Run with:

```
cd src-tauri
cargo test --lib
```

L3 — Rust integration tests

`src-tauri/tests/metadata_fs.rs` and `src-tauri/tests/auto_tag.rs` exercise the full pipeline on temporary directories:

Test	Verifies
`read_sidecar_end_to_end`	Legacy sidecar read: title + tags.
`read_sidecar_case_variants`	XMP parser is namespace/case insensitive.
`fts_delete_after_tag_update_works`	The FTS5 `contentless_delete` regression is fixed.
`batch_tag_add_persists_for_every_image`	Bulk `tags_add` / `tags_remove` semantics.
`embed_xmp_roundtrip_jpeg`	JPEG embed writes, re-reads, replaces on second write.
`embed_xmp_roundtrip_png`	PNG iTXt embed, no chunk stacking on rewrite.
`embed_xmp_roundtrip_webp`	WebP RIFF `XMP ` chunk roundtrip, no stacking.
`embed_xmp_roundtrip_gif`	GIF89a Application Extension XMP roundtrip.
`write_never_creates_sidecar_for_jpeg`	Saving on a JPEG creates zero `.xmp` files.
`write_removes_legacy_sidecar_after_embed`	A pre-existing `.xmp` is removed on successful embed.
`write_preserves_foreign_rating_and_description`	Foreign `xmp:Rating` and `dc:description` survive a tag edit.
`write_errors_on_unsupported_format`	Saving on RAW returns `Err`, no sidecar fallback.

Test	Verifies
`registry_recognises_every_expected_extension`	Every advertised handler is registered.

`auto_tag.rs` covers the automatic-AI-tagging DB primitives using the shipped [`MockClassifier`]:

Test	Verifies
`first_pass_tags_every_image`	Every candidate gets user tags + an `ai_tag_hash` on the row.
`second_pass_skips_unchanged_images`	Rerun on an unchanged folder is a no-op.
`changed_fingerprint_reclassifies`	Bumping `mtime_ms` invalidates the fingerprint and re-triggers.
`ai_tags_land_as_auto_and_coexist_with_scanner`	AI tags land under `'auto'` (never `'user'`); existing user tags and XMP-derived auto tags survive the pass without duplication.
`candidates_include_never_tagged_rows`	Migration path: NULL `ai_tag_hash` rows still surface.

Run:

```
cd src-tauri
cargo test --test metadata_fs
cargo test --test auto_tag
```

Testing the auto-tag pipeline end-to-end

`auto_tag.rs` integration tests use the deterministic `MockClassifier` so they don't need any ML dependencies. The production classifier is `ClipClassifier` (`core::auto_tag::clip_classifier`), which is exercised manually:

1. Launch Magpie and open (or create) a project.
2. **Settings** → **Auto-tag photos…** — the dialog opens. The **AI model** section reports *Not downloaded* on a fresh install. Click **Download AI model** and wait for the green progress bar to fill (~600 MB, one-time). Progress ticks come via `app://ai-model-download`.

3. Once the banner flips to *Model ready*, tick *Automatically tag photos when adding a new folder* and close the dialog.
4. Add a folder with a handful of JPEGs. The status bar shows the scan line first, then the auto-tag line (green bar).
5. Open one of the newly-scanned images. A handful of tags picked from `photo_vocab_v1.txt` (e.g. *beach*, *dog*, *portrait*) appear in **Automatic tags** (read-only, with the lock affordance), not in **Your tags**.
6. Re-add the same folder (after `remove_library_folder`) — the second run's status bar reports zero new tags because every row still matches its `ai_tag_hash` fingerprint.
7. Turn the toggle off in the dialog and add another folder: the scan runs but no `app://auto-tag` event fires and the DB's `ai_tagged_at` column stays NULL.

To force a re-download in a manual test, click **Remove model files** in the Auto-tag dialog — this deletes every file under `<app_data_dir>/models/clip/` (weights + tokenizer + cached vocab embeddings) and turns the toggle off.

If you add a folder before the model has finished downloading, `tag_folder` emits a single `AutoTagProgress { finished: true, error: Some("AI model not downloaded — …"), .. }`. The status bar renders that as a small amber warning and the DB row's `ai_tagged_at` stays NULL, so the next re-add or rescan can retry once the download completes.

L4 — screenshot smoke tests

Under `scripts/`:

- `screenshot-app.ps1` — launches Magpie, finds its window with `EnumWindows`, screenshots via `PrintWindow` (`PW_RENDERFULLCONTENT`) so the shot works even if another window is in front.
- `screenshot-scrolled.ps1` — clicks an image, scrolls the details panel, screenshots.
- `test-multiselect-tag.ps1` — end-to-end skeleton for Ctrl+click → type tag → click Apply. UI automation is inherently flaky on Windows; use these mostly to verify a build launches and paints, not for asserting behaviour.

These are not part of automated CI. They're one-off tools when investigating a UI-only bug.

Frontend tests

Currently minimal:

- Type safety is the main safety net.
- Component tests would use Vitest + React Testing Library; not set up in v1.

Because the UI is a thin translation over Rust commands, the interesting behaviour is on the Rust side and covered by L2/L3.

Format sample files

`samples/format-samples/` holds one average-sized synthetic file per handler registered in `FormatRegistry` (35 handlers, plus extension aliases such as `.jpeg`, `.tif`, `.m4v`).

Generate or refresh:

```
py -m pip install pillow reportlab pillow-heif imageio-ffmpeg
py samples/format-samples/generate_all.py
```

Output lands in `samples/format-samples/files/<ext>/sample.<ext>` with sizes and placeholder notes recorded in `manifest.json`. See `samples/format-samples/README.md` for caveats (camera RAW and some exotic codecs are scan-only placeholders, not vendor-native files).

Use these when manually exercising the scanner, thumbnail pipeline, or details panel against a broad extension set without copying a real library onto the machine.

Diagnostic tools (not tests, but adjacent)

- `src-tauri/examples/dump_meta.rs` — prints filesystem state, embedded XMP, any legacy sidecar contents, parsed metadata, and DB state for one photo or the first N rows in the DB. Great for reproducing bugs.
- `src-tauri/examples/dump_tag_usage.rs` — given a tag name, list every photo carrying it plus what the on-disk embedded XMP shows. Used in the “did the batch save actually work?” investigation.

Run:

```
cd src-tauri
cargo run --example dump_meta -- "C:\Photos\IMG_1234.jpg"
```

CI (recommended)

Not shipped in the repo yet, but the natural setup is:

- Windows runner (matrix: `windows-2022`).
- Steps: `cargo fmt --check`, `cargo clippy --all-targets`, `cargo test --workspace`, `npm ci`, `tsc -b`, `npm run tauri build -- --no-bundle`.
- Artifacts: `desktop.exe` for smoke inspection.

Implementation instructions — automatic AI tagging

> **Status:** Landed. Both the pipeline and the production `ClipClassifier` (candle-backed CLIP-ViT-B/32) are shipping. This document is kept as a design record — the "Phase 1 mock" section below describes the initial slice, the "Phase 2 — real CLIP" section at the end covers what actually ships today.

Copy this document (or link to it) when implementing automatic AI tagging in Magpie / PicOrg.

Project context

Magpie (repo: PicOrg) is a Tauri 2 desktop photo library app.

Layer	Stack
Backend	Rust — `src-tauri/src/`
Frontend	React 19 + TypeScript + TanStack Query + Zustand — `src/`
IPC	`src/ipc.ts` → Tauri `invoke`
Events	Backend emits Tauri events; frontend `listen`s
Tags	Stored in SQLite `magpie.db` only (not written to source files)
Tag writes	`batch_update_metadata(ids, { tagsAdd: [...] })`

Do not add a TopBar button. AI tagging is automatic and optional.

Revised requirements

1. **Automatic trigger:** When a library folder is added to a project, run AI tag assignment for that folder **after** the normal filesystem scan finishes.

2. **User control:** Add a **Settings** → **Auto-tag photos** menu item that toggles the feature on/off. Persist the preference in app settings.
3. **Non-blocking:** AI work runs in a background task (same pattern as `scanner::scan_folder`). The UI must remain fully interactive.
4. **Progress UI:** Show AI progress in the **bottom status bar** (`StatusBar.tsx`), mirroring the existing filesystem scan progress bar.

Architecture

```
add_library_folder()
  └─ spawn: scanner::scan_folder()     → app://scan
       └─ on success + aiAutoTag enabled:
            spawn: auto_tag::tag_folder() → app://auto-tag

Settings menu "Auto-tag photos" (checkable)
  └─ toggle aiAutoTag in app-settings.json
```

Concurrency rules:

- Filesystem scan and AI tagging must not block each other on the UI thread.
- Chain AI **after** scan completes so image rows and thumbnails exist.
- Use a semaphore inside `auto_tag` (like scanner) so inference does not saturate CPU/GPU.
- If the user adds several folders quickly, queue AI jobs per folder (FIFO) rather than running all in parallel.

Backend tasks

1. App settings — add `aiAutoTag`

Files: `src-tauri/src/core/project.rs`, `src-tauri/src/commands/settings.rs`, `src/types.ts`

Add to `AppSettings`:

```rust
#[serde(default = "default_ai_auto_tag")]
pub ai_auto_tag: bool,
```

Default: `false` (opt-in; first enable via menu should be enough — no separate consent dialog required for this slice unless product asks for one).

Extend `AppSettingsPatch` and `update_app_settings` to accept `ai_auto_tag: Option<bool>`.

Mirror in TypeScript `AppSettings` / `AppSettingsPatch`.

2. New module — `src-tauri/src/core/auto_tag/mod.rs`

Create `auto_tag::tag_folder()` with the same structural pattern as `scanner::scan_folder`:

```
pub const AUTO_TAG_EVENT: &str = "app://auto-tag";

pub async fn tag_folder(
    services: Arc<AppServices>,
    app_handle: AppHandle,
    folder_id: i64,
) -> AppResult<AutoTagResult>
```

Per-image loop:

1. Query image IDs in folder (`queries::…`, image kind only, not missing).
2. Skip if already AI-tagged and `content_hash` unchanged (add DB columns — see §5).
3. Load thumbnail bytes via existing thumbnail cache (`thumbnail::ensure_thumbnails` if needed).
4. Run classifier in `spawn_blocking` (stub OK for first PR — return 2–3 deterministic tags from a fixed vocabulary).
5. Filter by `min_confidence`, cap at `max_tags_per_image`.
6. `apply_metadata_patch({ tags_add })` in a transaction.
7. Update `ai_tagged_at` / `ai_tag_hash` on the image row.
8. Emit progress every image (or every 5 images if performance requires it).

Progress payload — add to `src-tauri/src/types.rs`:

```
pub struct AutoTagProgress {
    pub folder_id: i64,
    pub processed: i64,
    pub total: i64,
    pub current_path: Option<String>,
    pub tags_added: i64,      // cumulative tags written this run
    pub finished: bool,
}
```

3. Hook into folder add

File: `src-tauri/src/commands/library.rs` — `add_library_folder`

Replace the bare scan spawn with a chained task:

```
tauri::async_runtime::spawn(async move {
    match scanner::scan_folder(...).await {
        Ok(_) => {
            if services_bg.get_settings()?.ai_auto_tag {
                if let Err(e) = auto_tag::tag_folder(services_bg, app_handle_bg,
folder_id).await {
                    tracing::error!(error = %e, "auto-tag failed");
                }
            }
        }
        Err(e) => tracing::error!(error = %e, "scan failed"),
    }
});
```

Do not trigger AI on `rescan_folder` / `rescan_all` in this slice unless explicitly requested.

4. Optional: AI job queue

If multiple folders are added in quick succession, use a simple mutex-protected queue in `AppServices` or a dedicated `AutoTagScheduler` so only one folder is AI-tagged at a time (scan can still run in parallel per folder).

5. Database migration

File: new migration in `src-tauri/src/db/migrations/` (follow existing pattern)

```
ALTER TABLE images ADD COLUMN ai_tagged_at INTEGER;
ALTER TABLE images ADD COLUMN ai_tag_hash TEXT;
```

Update `docs/developer/src/design/schema.md`.

6. Classifier stub (Phase 1)

File: `src-tauri/src/core/auto_tag/classifier.rs`

For the first PR, implement a **mock classifier** that returns tags from a fixed vocabulary based on image id hash. Structure the trait so a real ONNX/CLIP sidecar can replace it later:

```
pub trait ImageClassifier: Send + Sync {
    fn classify(&self, image_bytes: &[u8]) -> AppResult<Vec<TagSuggestion>>;
}
```

No bundled ML model required in this slice.

7. Register commands / modules

- Add `pub mod auto_tag;` in `src-tauri/src/core/mod.rs`
- No new frontend-invokable command is strictly required if AI is fully automatic; settings toggle uses existing `update_app_settings`.
- Document any new types in `docs/developer/src/design/commands.md` (events section).

Frontend tasks

1. Menu — toggle item

File: `src-tauri/src/menu.rs`

Add to **Settings** submenu:

```
pub const ID_SETTINGS_AI_AUTO_TAG: &str = "set_ai_auto_tag";
```

Use a **checkable** menu item (`MenuItemBuilder::…` with check state). On build, read current `ai_auto_tag` from settings if available, or default unchecked.

Tauri note: if native checkmarks are awkward at build time, use a plain item whose label reflects state (`Auto-tag photos ✓` / `Auto-tag photos`) and sync on settings load.

File: `src/App.tsx` — extend `useMenuRouter`:

```
case 'set_ai_auto_tag':
  return h.onToggleAiAutoTag()
```

Handler:

1. Read current settings from Zustand / React Query cache.
2. Call `updateAppSettings({ aiAutoTag: !current })`.
3. Update Zustand + invalidate `['settings']`.
4. Call new helper `setMenuItemChecked('set_ai_auto_tag', enabled)` if implemented, or relabel via a new Tauri command.

Add optional Tauri command `set_menu_item_checked(id, checked)` alongside existing `set_menu_item_enabled` in `menu.rs`.

Sync menu check state when app loads settings (in `App.tsx` after `getAppSettings` resolves).

2. IPC — auto-tag progress event

File: `src/ipc.ts`

```
export const onAutoTagProgress = (handler: (p: AutoTagProgress) => void) =>
  listen<AutoTagProgress>('app://auto-tag', (e) => handler(e.payload))
```

Add `AutoTagProgress` to `src/types.ts`.

3. Status bar — dual progress display

File: `src/features/StatusBar.tsx`

Currently listens only to `app://scan`. Extend to also listen to `app://auto-tag`.

Display rules:

State	Status bar shows
Scan running	Existing: `Scanning — N / M` + progress bar
AI running	`Auto-tagging — N / M` + progress bar + optional current path
Both running	Show **both** lines stacked, or combine: `Scanning… · Auto-tagging…` with two thin bars
Finished	`Auto-tag complete` (auto-hide after 2.5 s, same as scan)

Reuse existing Tailwind progress bar markup from scan.

On `finished: true` for auto-tag, invalidate React Query keys: `['images']`, `['tags']`.

4. Remove / do not add TopBar button

Do **not** add an Auto-tag button to `TopBar.tsx`.

5. Settings dialog (optional)

If `SettingsDialogs.tsx` exists for theme/font, optionally add a checkbox there too — menu toggle is sufficient for this slice.

Documentation (required by repo rules)

Update in the same PR:

Doc	Change
`docs/user-manual/`	New short section: “Automatic tagging” — explain Settings menu toggle, background behaviour, status bar progress, tags go to library only
`docs/developer/src/design/testing.md`	How to test auto-tag with mock classifier
`docs/developer/src/design/schema.md`	New columns
`docs/developer/src/design/commands.md`	`app://auto-tag` event payload
`docs/developer/src/functional/out-of-scope.md`	Move auto-tagging from “out of scope” to “implemented (opt-in, local stub)” or similar

Run: `npm run docs:build`

Use plain language in the user manual (no jargon like “ONNX”, “spawn_blocking”).

Testing checklist

Rust integration test (`src-tauri/tests/`):

1. Add folder with mock classifier enabled → images receive `tagsAdd` entries.

2. Add folder with `aiAutoTag: false` → no tags added by AI.
3. Re-add / unchanged hash → image skipped on second AI run.

Manual smoke:

1. Settings → Auto-tag photos → checkmark appears.
2. Add folder → status bar shows scan, then auto-tag progress.
3. UI remains responsive (scroll grid, open details) during auto-tag.
4. Tags appear in sidebar tag list after completion.

File touch list (summary)

```
src-tauri/src/core/auto_tag/mod.rs          (new)
src-tauri/src/core/auto_tag/classifier.rs   (new, mock)
src-tauri/src/core/mod.rs
src-tauri/src/core/project.rs
src-tauri/src/commands/library.rs
src-tauri/src/commands/settings.rs
src-tauri/src/menu.rs
src-tauri/src/types.rs
src-tauri/src/db/migrations/NNN_ai_tag.sql  (new)
src-tauri/src/db/queries.rs                 (skip logic, column updates)
src/types.ts
src/ipc.ts
src/App.tsx
src/features/StatusBar.tsx
docs/…
```

Explicit non-goals for this slice

- No TopBar button
- No cloud API
- No real ML model bundle (mock classifier only)
- No AI on manual rescan
- No writing tags into source files
- No cancel button (can be a follow-up)

Suggested PR title

`feat: opt-in automatic AI tagging after folder add with status bar progress`

Reference implementation patterns

Read these files first, then mirror their patterns:

- `src-tauri/src/core/scanner.rs` — background job, semaphore, progress events
- `src-tauri/src/commands/library.rs` — `add_library_folder` spawn hook
- `src/features/StatusBar.tsx` — scan progress bar UI
- `src/App.tsx` — `useMenuRouter` for Settings menu actions
- `src-tauri/src/menu.rs` — native menu item IDs and build

Phase 2 — real CLIP (delivered)

Phase 1 shipped the mock classifier described above. Phase 2 replaced it with an on-device model. Two moving parts land alongside the existing pipeline:

Classifier — `core/auto_tag/clip_classifier.rs`

- Uses `candle` (https://github.com/huggingface/candle)'s pure-Rust CLIP implementation (`candle_transformers::models::clip`). Chosen over ONNX Runtime because the `ort-sys` prebuilt-binary CDN (`pyke.io`) proved unreliable during integration and candle removes the whole "download prebuilt native lib" failure mode.
- Model: **OpenAI CLIP-ViT-B/32**. Runs on `Device::Cpu`; ~150-300 ms per image on a modern laptop, which is fine for the folder-add flow.
- Preprocessing (`preprocess_image`) mirrors OpenAI's reference pipeline: resize the shorter side to 224 px, centre-crop 224×224, normalise with `CLIP_MEAN` / `CLIP_STD`, laid out as NCHW `[1, 3, 224, 224]`.
- Vocabulary: `core/auto_tag/resources/photo_vocab_v1.txt` — ~1 000 photo words (scenes, objects, animals, activities, colours, lighting) as a plain UTF-8 file, one entry per line, `#` comments allowed. Bump `VOCAB_VERSION` in `model_manager.rs` when the file changes; the on-disk embedding cache is keyed by `SHA-256(VOCAB_TEXT)` so old caches invalidate automatically.

- Ranking: cosine similarity (both sides L2-normalised → dot product) between the 512-dim image embedding and every row of the cached text-embedding matrix. Keep top-`MAX_TAGS_PER_IMAGE=6` above `MIN_COSINE=0.20`.
- Text embeddings for the vocab are computed once via `compute_text_embeddings` (tokenise `"a photo of a <tag>"` with the CLIP BPE tokenizer, pad to 77, run `get_text_features`, L2-normalise) and cached under `<app_data_dir>/models/clip/photo_vocab_v1.embeddings.f32` as packed `f32` (via `bytemuck::cast_slice`).

Model files — `core/auto_tag/model_manager.rs`

- Two files are downloaded lazily on first use from HuggingFace and cached under `<app_data_dir>/models/clip/`:
  - `model.safetensors` (~605 MB, pinned SHA-256).
  - `tokenizer.json` (~2 MB, trusted).
- Downloads stream chunk-by-chunk via `reqwest` + `futures-util`, written to a `.part` file, SHA-verified, then atomically renamed.
- **TLS uses the Windows trust store** (`reqwest`'s `rustls-tls-native-roots` feature → `rustls-native-certs` → SChannel). Bundled Mozilla roots weren't enough on corporate networks that MITM outbound HTTPS.
- **Resumable and retried.** Each file is attempted up to 4 times with 2/4/8 s exponential backoff. Retries send `Range: bytes=N-` from the current `.part` size so a mid-stream drop costs one round-trip, not a full re-download. The final SHA-256 covers whatever bytes were already on disk plus every new chunk.
- **Full error chains.** `chain(&err)` walks `Error::source()` recursively so the dialog shows the actual cause (TLS handshake, DNS lookup, mid-stream read timeout, …) instead of the top-level "error sending request" reqwest wraps everything in.
- Progress events fire on `app://ai-model-download` throttled to 200 ms. `AppServices.auto_tag_gate` (also used by `tag_folder`) serialises downloads against any concurrent AI pass.
- `check_status(app_data_dir)` is a cheap `stat`-only probe and never blocks the UI thread.

Commands and UI

- New commands `check_ai_model_status`, `download_ai_model`, `clear_ai_model` — see `commands.md → AI-model management commands`.
- Menu entry became **Settings → Auto-tag photos…** — a plain item that opens `AiAutoTagDialog` in `src/features/SettingsDialogs.tsx`. The dialog owns the

enable/disable checkbox (kept disabled until the model is `ready`) and the download / remove buttons.
- The status bar surfaces a small warning if a scan finishes but the auto-tag pass couldn't start because the model isn't on disk.

Vocabulary and tuning knobs

Everything user-facing lives in `photo_vocab_v1.txt`. To retune:

1. Edit the vocab. Keep entries short common nouns; the prompt template `"a photo of a "` is prepended so multi-word entries like `"black and white photo"` work but should stay natural English.
2. Bump `VOCAB_VERSION` in `model_manager.rs`.
3. `MIN_COSINE` and `MAX_TAGS_PER_IMAGE` in `clip_classifier.rs` are the two other knobs. Lower `MIN_COSINE` produces noisier tags; raise it for higher-precision but sparser results.

Testing

Integration tests in `src-tauri/tests/auto_tag.rs` still use the `MockClassifier`, injected through `tag_folder_with`. The `ClipClassifier` isn't exercised in CI because the ~600 MB model weights aren't downloaded there — the manual test procedure lives in `testing.md`.

Build and release

Prerequisites (Windows)

```
# Rust toolchain (user scope)
winget install --id Rustlang.Rustup --accept-source-agreements

# MSVC C++ Build Tools (machine scope; needs UAC)
winget install --id Microsoft.VisualStudio.2022.BuildTools `
  --override "--wait --passive --add Microsoft.VisualStudio.Workload.VCTools --
includeRecommended"

# Node LTS
winget install --id OpenJS.NodeJS.LTS
```

Verify:

```
rustc --version   # ≥ 1.77.2
cargo --version
node --version    # ≥ 18
npm --version
```

Clone and build

```
git clone <repo> magpie
cd magpie
npm install
```

Development

```
npm run tauri dev
```

This starts:

- **Vite dev server** on `localhost:1420` with HMR.
- **Cargo watch** compiling the Rust core.
- **Tauri window** loading the dev server URL.

Rust code changes trigger a full app rebuild (10–30 s); frontend changes hot-reload in <1 s.

Release build

```
npm run tauri build -- --no-bundle
```

- `--no-bundle` skips the MSI/NSIS installer, producing just `src-tauri/target/release/desktop.exe`. Faster and useful for local testing.
- Remove `--no-bundle` (or pass `--bundles msi` / `--bundles nsis`) to produce an installer under `src-tauri/target/release/bundle/`.

Timings on a modern laptop:

- Cold release build: ~5 min.
- Incremental release build after a small Rust change: ~90 s.
- Incremental after frontend change only: ~30 s.

Tauri configuration

Key fields in `src-tauri/tauri.conf.json`:

Field	Notes
`productName`	`Magpie`
`identifier`	`com.magpie.app` — used for `%APPDATA%` folder name.
`security.csp`	Strict; blocks inline scripts and remote origins.
`app.withGlobalTauri`	`false` — commands accessed via `@tauri-apps/api` only.
`bundle.icon`	Multi-size Windows ICO.
`bundle.windows.wix`	(Present in future for signed MSI.)

Capabilities (`src-tauri/capabilities/default.json`) declare only what's needed: dialog and opener plugins, asset access under the user's library folders. Network is *not* granted.

Release profile

`src-tauri/Cargo.toml`:

```
[profile.release]
codegen-units = 16
lto           = false
opt-level     = 3
panic         = "abort"
strip         = true
incremental   = false
```

Rationale:

- `codegen-units = 16` + `lto = false` — trades a bit of runtime perf for much faster incremental builds. In practice, the hot paths are limited by `image` decode and disk I/O, not by cross-crate inlining.
- `panic = "abort"` — smaller binary, no unwinding tables. All panics are treated as bugs; the app crashes and restarts.
- `strip = true` — removes debug symbols to keep the binary small (~13 MB stripped vs. 45 MB with symbols).

Version bump

1. `Cargo.toml`: `version = "0.1.1"` (both files if we ever add a workspace).
2. `package.json`: same.
3. `tauri.conf.json`: `version = "0.1.1"`.
4. Commit as `chore: v0.1.1`.

Releasing

The v1 workflow is manual:

1. `npm run tauri build` (with default bundle) on a clean tree.
2. Test the resulting MSI on a clean VM.
3. Draft a GitHub release with the MSI attached.
4. Attach the two PDFs from `docs/` as documentation assets.
5. Tag the commit `v0.1.0` and publish.

A GitHub Actions workflow is not shipped in v1 but can plug into the recommended CI setup in Testing strategy.